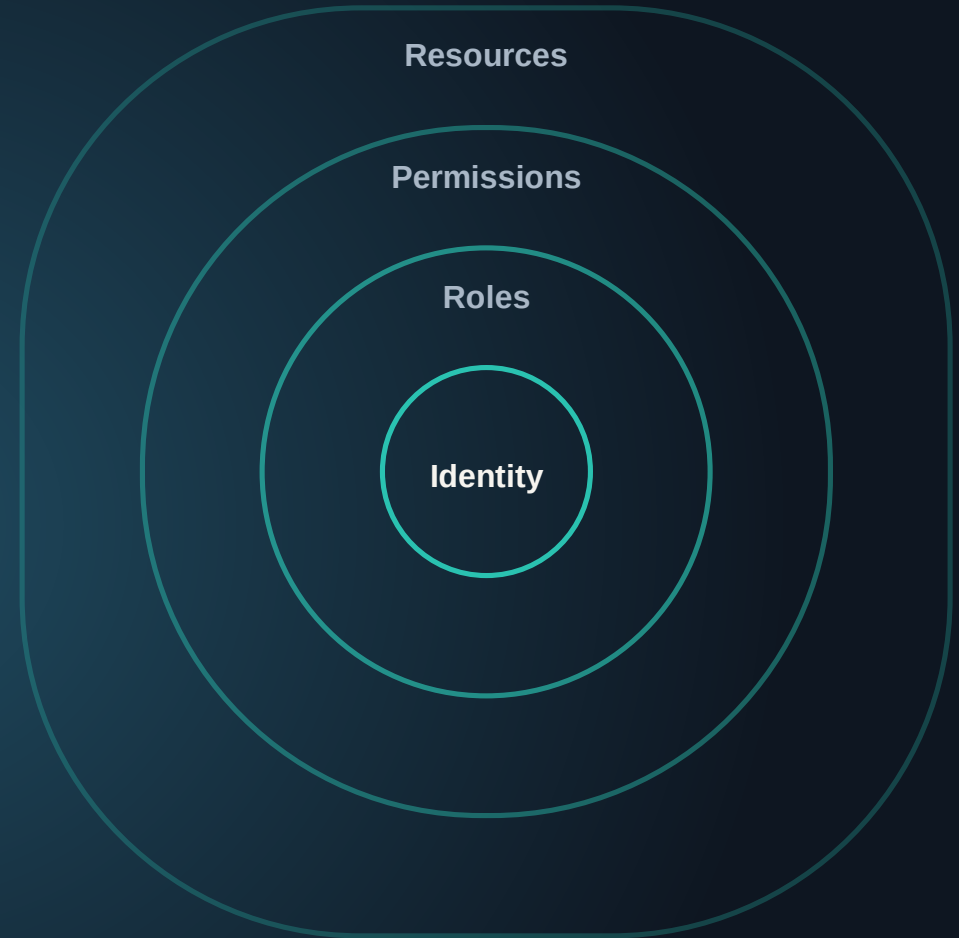


TECHNICAL CURRICULUM

Identity & Access Management and Least Privilege in Enterprise Systems

From identity fundamentals and protocols to cloud entitlements, privileged access, governance and identity threat detection.



ORIENTATION

What you will be able to do

Explain

- Trace an access request end to end: proofing, authentication, token, policy decision, audit
- Read protocol traffic (SAML, OAuth, OIDC, Kerberos) and spot flaws

Design

- Choose between RBAC, ABAC and ReBAC
- Design admin tiering, JIT elevation, guardrails and federation trust

Implement

- Write AWS, Azure and GCP policies, OPA and Cedar rules
- Replace long-lived keys with workload identity federation

Operate

- Run access reviews and measure the permission gap
- Detect identity attacks and respond to token theft

Format 12 parts, each closing with a checkpoint. Slides carry the structure; speaker notes carry deeper commentary.

ORIENTATION

Curriculum map

Part	Theme	You will master
1 Foundations	Concepts and architecture	Principals, credentials, PEP/PDP, Zero Trust, attack chain
2 Authentication	Proving who you are	Passwords, MFA, WebAuthn, Kerberos, sessions, recovery
3 Federation	Trust across boundaries	SAML, OAuth 2.0, OIDC, JWT, SCIM, LDAP, CAEP
4 Authorization	Deciding what is allowed	RBAC, ABAC, ReBAC, OPA, Cedar, Zanzibar, SoD
5 Least privilege	The core discipline	Blast radius, right-sizing, JIT, ZSP, tiering, KPIs
6 PAM and secrets	Protecting the keys	Vaulting, sessions, JEA, dynamic secrets, SPIFFE
7 Cloud IAM	AWS, Azure, GCP, Kubernetes	Policy evaluation, SCPs, PIM, WIF, escalation paths
8 Directory and hybrid	Active Directory and SSO	Tiering, Kerberoasting, hybrid sync, IdP resilience
9 Governance	Access right over time	JML, certification, SoD, compliance, RACI
10 Detection	Identity threat response	Telemetry, ATT&CK, KQL, token-theft playbooks
11 Programme	Making it real	Roadmap, target architecture, migration, case studies
12 Labs and reference	Practice	Six labs, checklists, glossary, standards

HOW TO READ THIS DECK

Textbox legend and notation

Key idea

A principle or takeaway to remember.

Watch out

A trap, limit, or common misconception.

Attack / risk

How an adversary abuses the design.

Annotation

A margin note that labels part of a diagram.

Code panels Red-highlighted lines are anti-patterns, teal-highlighted lines are the fix, amber lines deserve review. Grey lines are comments.

PART 1

Foundations

The vocabulary, architecture and threat model that every later part builds on.

IN THIS PART

- Vocabulary and the four questions
- Domain map and identity types
- Lifecycle and reference architecture
- Zero Trust and the identity attack chain
- Standards you will be measured against

WHY THIS MATTERS

Identity is the new perimeter.

Users, SaaS, APIs, workloads and agents reach resources over networks you do not own. What stays constant is who or what is asking, and what it is allowed to do.

Three shifts made identity the control plane

The network edge dissolved

SaaS, multi-cloud, remote work and partner APIs mean no single point where a firewall sees all access.

Attackers log in

Stolen passwords, session tokens, OAuth grants and over-privileged service accounts look like legitimate access.

Every action is a decision

Each cloud API call is authenticated and authorized. The policy set is the security posture.

Implication Every IAM design choice is a decision about blast radius: how much damage one compromised identity can do.

Identities and credentials

Term	Meaning	Example
Identity	Set of attributes describing an entity within a system	Employee record: ID, department, manager
Principal	An entity that can be authenticated and can act	User, role, service account, workload
Account	The identity as represented inside one system	AD account jdoe, AWS IAM role
Credential	Evidence used to prove an identity claim	Password, private key, certificate, token
Session	A bounded period of authenticated interaction	Browser cookie, Kerberos ticket
Token	Signed or opaque artifact carrying authn or authz facts	JWT, SAML assertion, opaque bearer

Access and policy

Term	Meaning	Example
Permission	An allowed action on a type of resource	s3:GetObject, db.select
Privilege	A permission with elevated impact, or the right to bypass controls	root, Domain Admin, Owner
Entitlement	A grant that links a principal to permissions	Role assignment, group membership
Role	A named bundle of permissions	BillingReader, PlatformOperator
Policy	Rules that yield allow or deny for a request	IAM JSON, Rego, Cedar
Scope	The boundary of a grant: resources, tenants, or OAuth strings	Subscription, project, scope=read:orders
Claim	An attribute asserted inside a token	groups, amr, tid

THE FOUR A'S

Administration, authentication, authorization, audit

Administration

Create, change and remove identities and entitlements.
Question: who should exist and what should they hold?

Authentication

Establish that the claimant controls the claimed identity.
Question: are you who you say you are?

Authorization

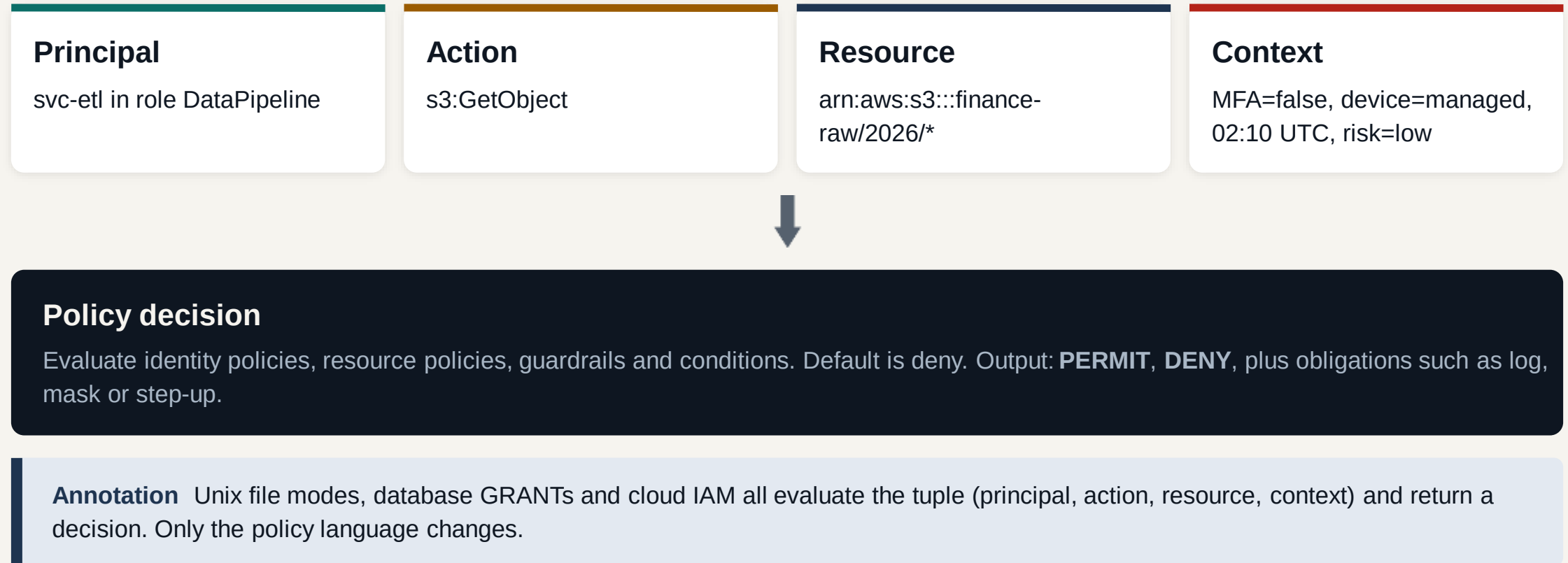
Decide whether a principal may do an action on a resource now. Question: are you allowed to do this, here, now?

Audit

Record who did what, when, from where and under what authority. Question: can we reconstruct and prove it?

Failure modes differ Authentication failure means impersonation. Authorization failure means privilege abuse. Audit failure means no forensics. Administration failure means all three accumulate silently.

Every authorization asks the same question



The IAM domain in six layers

Identity administration	Lifecycle, provisioning, directories, access requests, certification (IdM and IGA)
Authentication	SSO, MFA, federation, session management, customer identity (AM and CIAM)
Authorization	RBAC, ABAC, ReBAC, policy engines, API and data-layer enforcement
Privileged access	Vaulting, JIT elevation, session control, secrets and machine identity (PAM)
Cloud entitlements	Effective permissions, unused access, escalation paths (CIEM)
Threat detection	Telemetry, detections, response, deception (ITDR)
Reading order Prove identity, express trust, decide access, minimise it, protect it, govern it, detect abuse.	

Six populations, six lifecycles

Workforce

Employees and contractors. HR is the source of truth. Lifecycle JML, SSO and MFA.

Customer (CIAM)

Consumers and B2C. Scale, registration, consent, fraud and privacy regulation.

Partner (B2B)

External organisations. Federation, guest accounts, sponsors and expiry.

Workload

Services, jobs and containers. Certificates, managed identities, SPIFFE IDs.

Automation

CI/CD, IaC runners, bots. High privilege by nature. Prefer OIDC federation to keys.

AI agents

Tools acting through APIs on delegated authority. Scoped, short-lived, auditable.

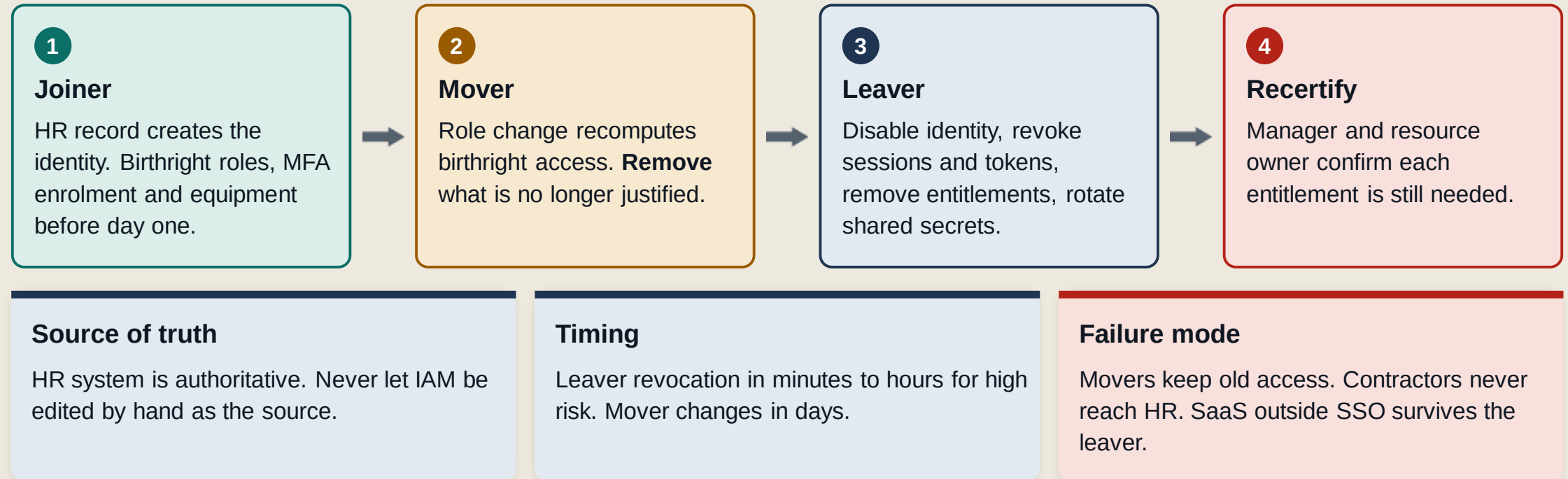
Watch out Non-human identities typically outnumber human ones in cloud-native estates yet often receive weaker lifecycle governance and longer-lived credentials.

Where machine credentials fail

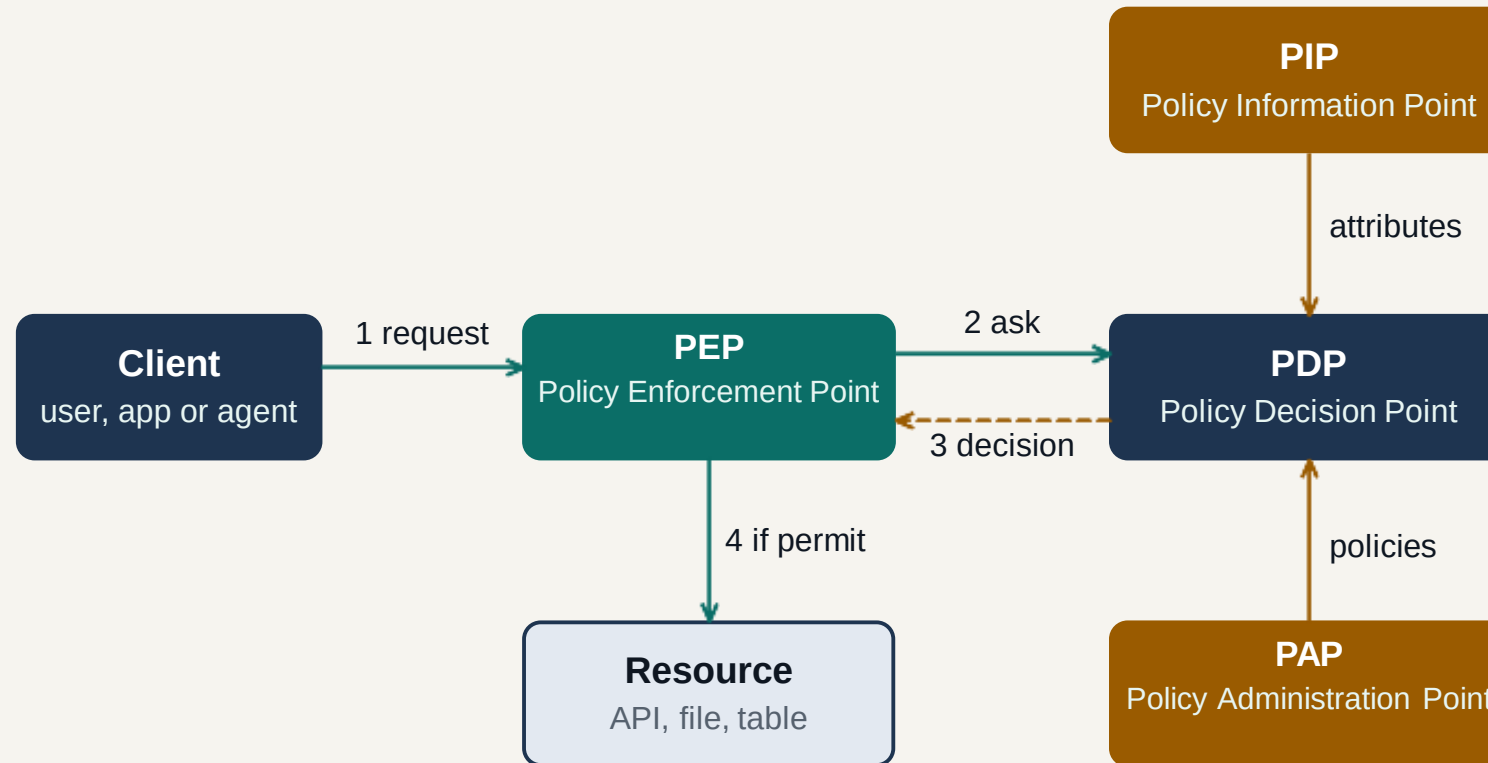
Type	Typical credential	Common failure	Control
AD service account	Password, Kerberos keys	Never expires, shared, Kerberoastable	gMSA, owner, rotation
Cloud access key	Long-lived key pair	Leaked in repo, never rotated	Roles, federation, no keys
Instance or pod role	Platform-issued token	SSRF to metadata, broad role	IMDSv2, scoped role
CI/CD secret	PAT, API key	Stored in pipeline vars, over-scoped	OIDC federation, short TTL
OAuth app	Client secret, cert	Broad consent, orphaned owner	Owner, scope review, cert auth

Rule of thumb If a credential can be copied and used from anywhere, treat it as a liability. Prefer identities that are issued at runtime, short-lived and bound to the workload.

Joiner, mover, leaver, recertify



PEP, PDP, PIP and PAP working together



PEP

Enforces only. Contains no decision logic. **Fails closed** when the PDP is unreachable.

PDP

Stateless evaluation of policy against attributes. Default deny. Returns obligations.

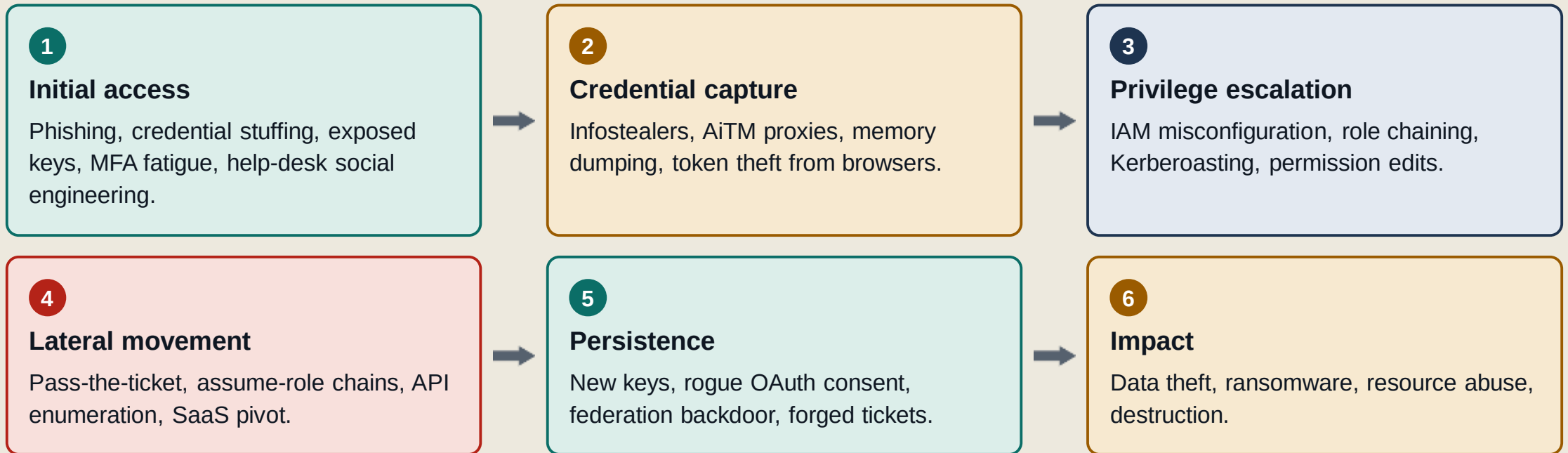
PIP and PAP

PIP supplies roles, device posture and risk. PAP is where policies are authored, tested and versioned.

NIST SP 800-207 tenets and their IAM meaning

Tenet	What it means for IAM
1. All data sources and services are resources	Give workloads, SaaS and APIs identities, not only users
2. All communication is secured regardless of location	No implicit trust from network position
3. Access is granted per session	Short-lived credentials, re-evaluation on each session
4. Access is decided by dynamic policy	Identity, device, behaviour and environment as inputs
5. Asset integrity and posture are monitored	Device compliance feeds access decisions
6. Authn and authz are strictly enforced before access	PEP and PDP on every path, dynamic re-evaluation
7. Collect telemetry to improve posture	Identity logs drive detection and policy tuning

The identity-centric attack chain



Where least privilege bites It caps what a stolen identity can reach and how far it can escalate. Steps 3 to 5 depend on excess permission.

Frameworks you will be measured against

Framework	Relevant controls	Use it for
NIST SP 800-63-4	Proofing, authenticators, federation	Assurance levels (IAL, AAL, FAL)
NIST SP 800-53 Rev. 5	AC-2, AC-3, AC-5, AC-6, IA-2, IA-5, AU	Control baselines and audit
NIST SP 800-207	Zero Trust Architecture	PDP and PEP design
ISO/IEC 27001:2022	Annex A 5.15 to 5.18, 8.2, 8.3, 8.5	ISMS certification
CIS Controls v8	5 Account Management, 6 Access Control	Prioritised safeguards
PCI DSS v4.0	Requirements 7 and 8	Cardholder data environments
SOC 2 Trust Services	CC6.1 to CC6.3	Logical access assurance
OWASP	ASVS V2 to V4, API Top 10 (BOLA)	Application authn and authz testing

CHECKPOINT 1

Check your understanding

Question	Model answer
What is the difference between an identity, a principal and an account?	Identity: attributes of an entity. Principal: an entity that can authenticate and act. Account: its representation inside one system.
Name the four elements of the request tuple.	Principal, action, resource, context. The output is a decision.
Which component enforces and which decides?	PEP enforces, PDP decides. PIP feeds attributes, PAP manages policy.
Why does the mover event cause permission creep?	Access is added for the new role but the old role's access is not removed.
Which attack-chain stages does least privilege limit most?	Privilege escalation, lateral movement and persistence.

PART 2

Authentication

How systems establish that a claimant controls an identity, and how attackers defeat that.

IN THIS PART

- Factors, passwords and storage
- MFA strength and phishing resistance
- TOTP and WebAuthn internals
- Assurance levels and adaptive policy
- Sessions, token theft, Kerberos, NTLM
- Account recovery: the weakest link

FACTORS

Authentication factors and how each one fails

Factor	Examples	Strength	Typical failure
Knowledge	Password, PIN	Weak alone	Phished, reused, guessed, stuffed
Possession	FIDO2 key, smart card, phone app	Strong if non-exportable	Stolen device, SIM swap, push fatigue
Inherence	Fingerprint, face	Local unlock convenience	Not secret, presentation attacks, cannot rotate
Context	IP reputation, device health, geo, time	A signal, not proof	Spoofed by proxies, false positives
Key binding	TPM-bound key, passkey	Phishing resistant	Recovery path becomes the weak link

Key idea Multi-factor means factors of different categories. Two passwords are not MFA. Biometrics typically unlock a local key rather than being sent to the server.

Current guidance: NIST SP 800-63B-4

Length over complexity

- At least 15 characters when the password is the only factor, 8 as part of MFA
- Allow 64 or more, all printable Unicode and spaces
- No composition rules

No forced rotation

- Change on evidence of compromise only
- Periodic expiry breeds predictable increments such as Summer2026!

Screen and throttle

- Block breached, common and context-specific values
- Allow paste and password managers
- Rate-limit guesses per account and per source

Direction of travel The best password is none. Move the workforce to passkeys and phishing-resistant MFA, and keep passwords only as a fallback with compensating controls.

Storing passwords: slow, salted, memory-hard

Choose the KDF

- Argon2id m=19 MiB, t=2, p=1
- scrypt N=2¹⁷, r=8, p=1
- bcrypt cost 10+ (72-byte cap)
- PBKDF2-SHA256 600k+ (FIPS only)

Never

- MD5, SHA-1 or bare SHA-256
- Reversible encryption
- Unsalted or one global salt

python

```
# OWASP-aligned minimums for Argon2id
from argon2 import PasswordHasher

ph = PasswordHasher(
    time_cost=2,      # iterations
    memory_cost=19456, # KiB, about 19 MiB
    parallelism=1,
)

digest = ph.hash(password) # salt inside
ph.verify(digest, attempt) # raises on error
if ph.check_needs_rehash(digest):
    digest = ph.hash(attempt) # upgrade
```

Credential attacks and the control that stops each

Attack	How it works	Best defence
Phishing or AiTM proxy	Victim signs in to a look-alike; proxy relays credentials and captures the session	FIDO2 origin binding, compliant-device policy
Credential stuffing	Breached pairs replayed across services	Breach screening, bot defence, MFA
Password spraying	A few common passwords across many accounts	Smart lockout, block legacy auth, low-and-slow detection
Offline cracking	Guessing against a stolen hash	Memory-hard KDF, long passphrases
MFA fatigue	Repeated push prompts until approval	Number matching, prompt limits, phishing-resistant MFA
Session or token theft	Infostealer or AiTM copies the cookie or token	Short lifetimes, device-bound tokens, CAE
Help-desk social engineering	Attacker impersonates staff to reset MFA	Strong verification, callback, alerting

MFA methods ranked by phishing resistance

Method	Phishing resistant	Notes
SMS or voice OTP	No	SIM swap, SS7 interception; NIST treats as restricted
Email OTP or magic link	No	Only as strong as the mailbox
TOTP authenticator app	No	Shared secret; real-time relay works
Push approve or deny	No	Fatigue and blind approval
Push with number matching and context	Partial	Reduces blind approval, still relayable
FIDO2 security key or passkey	Yes	Origin-bound public-key credential
Smart card or PIV (PKI)	Yes	Private key on card, PKINIT or mutual TLS
Windows Hello for Business	Yes	TPM-bound key with local PIN or biometric

Priority order Protect administrators and remote-access paths first with phishing-resistant methods, then expand to all users.

Inside a one-time code: HOTP and TOTP

Inputs

- Secret K 128+ bits (base32 in QR)
- Step 30 s, T0 = 0, 6 digits
- HMAC-SHA1 (SHA-256/512 allowed)

Server checks and limits

- Accept plus or minus 1 step for drift
- Store last counter to block replay
- Phishable: no origin binding

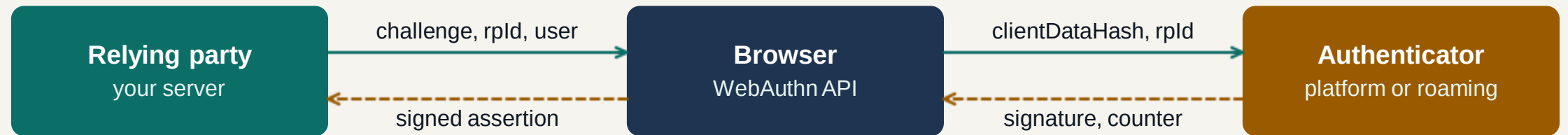
python (RFC 4226 / 6238)

```
import hmac, hashlib, struct, time

def hotp(key, counter, digits=6):
    msg = struct.pack(">Q", counter)
    h = hmac.new(key, msg, hashlib.sha1).digest()
    off = h[-1] & 0x0F # dynamic truncation
    b = h[off:off+4]
    code = struct.unpack(">I", b)[0] & 0x7FFFFFFF
    return str(code % 10**digits).zfill(digits)

def totp(key, step=30, t0=0):
    counter = int((time.time() - t0) // step)
    return hotp(key, counter) # RFC 6238
```

FIDO2 and WebAuthn architecture



Relying party stores

credentialId, **public key**, signCount, AAGUID. Never sees the private key or biometric.

Browser binds the origin

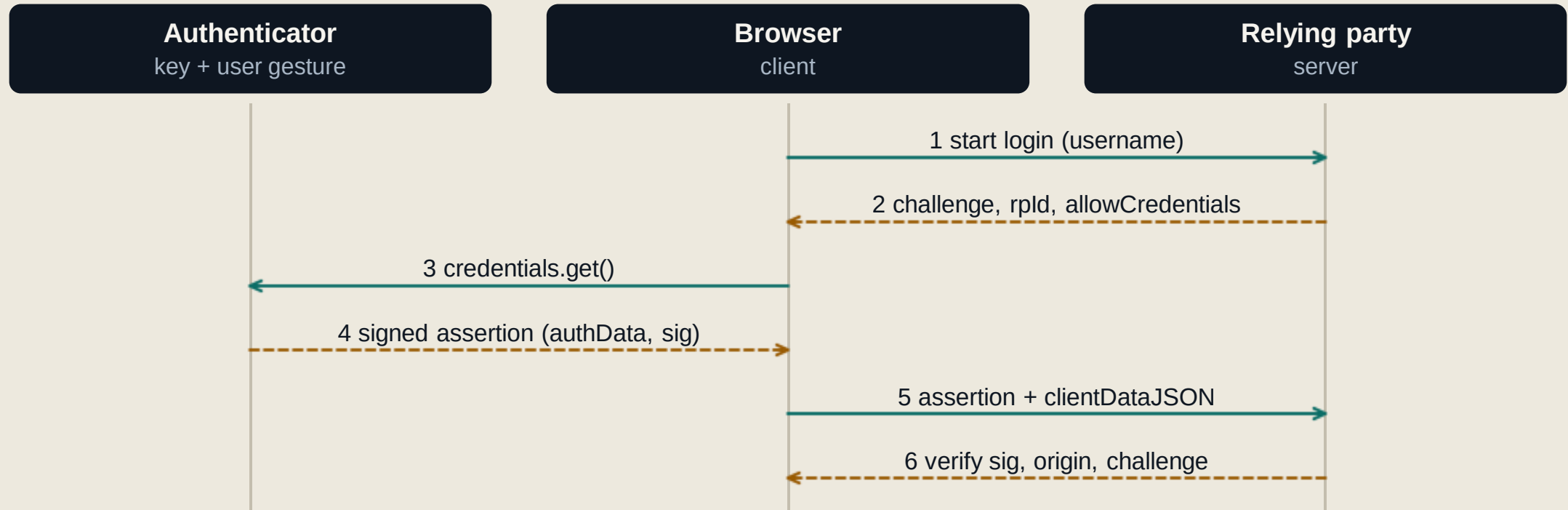
clientDataJSON carries the real **origin**. A look-alike domain cannot obtain a valid assertion.

Authenticator signs

Per-site private key. Requires user presence or verification. Key stays on device, or in the sync fabric for passkeys.

Annotation Registration creates a key pair per relying party. Authentication signs a fresh server challenge. There is no shared secret to steal from the server.

Authentication ceremony step by step



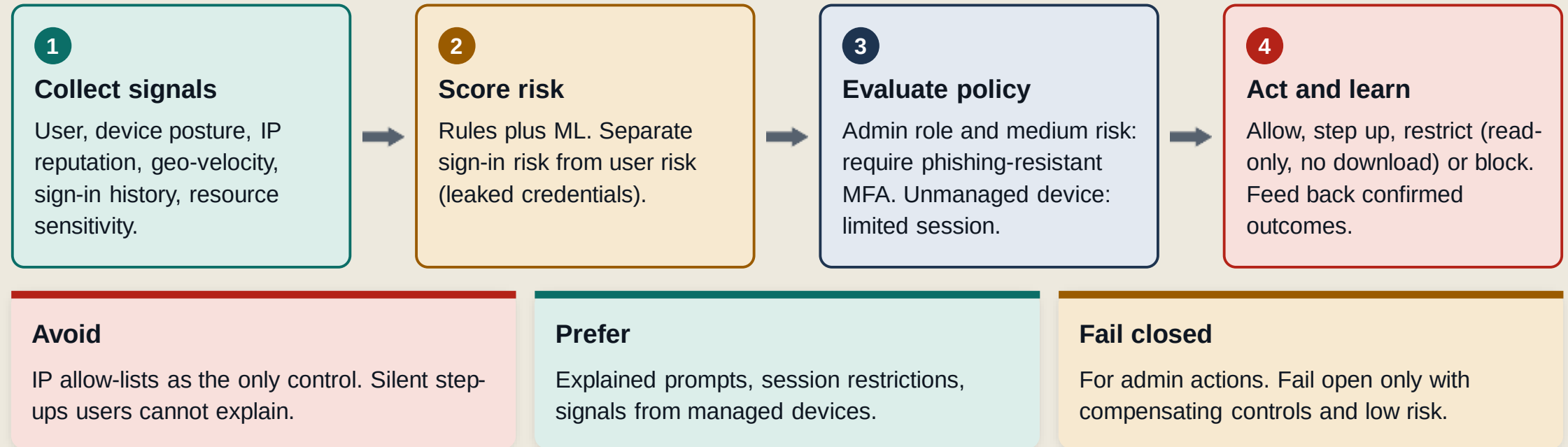
Why it resists phishing The browser signs the true origin, so an assertion made for evil.example is useless at bank.example.

NIST authenticator assurance levels

Level	What it demands	Typical authenticators	Use for
AAL1	Some confidence; single factor allowed	Password, OTP	Low-risk applications
AAL2	Two distinct factors, approved cryptography, replay resistance	Passkey, password + TOTP, platform key	Most workforce and customer apps
AAL3	Hardware-bound key, verifier impersonation resistance (phishing resistant)	FIDO2 security key, PIV smart card	Administrators, sensitive data

Related scales IAL measures how strongly identity was proofed. FAL measures how strongly a federation assertion is protected (signed, encrypted, bound to the relying party).

Risk-based authentication loop



Session cookies and lifecycle

Lifecycle

- 128-bit+ random ID (CSPRNG)
- Regenerate on login and privilege change
- Idle and absolute timeouts, revoke

Attacks

- Fixation: attacker sets the ID
- Hijack via XSS or infostealer
- CSRF: cookie sent cross-site

response headers

HTTP/1.1 200 OK

Set-Cookie: __Host-sid=8f3c9a...e91;

Path=/; Secure; HttpOnly;

SameSite=Lax; Max-Age=1800

__Host- prefix: Secure, Path=/, no Domain

HttpOnly: JavaScript cannot read it

SameSite: limits cross-site sending

Idle timeout 15-30 min, absolute 8-12 h

Bind to server state; revoke on logout,

password change, or risk event

Watch out Long-lived "remember me" cookies are bearer credentials. Rotate, bind and revoke them like refresh tokens. For SPAs prefer a backend-for-frontend with an HttpOnly cookie.

Assume bearer artifacts will be stolen

AiTM phishing

A reverse proxy relays the login including MFA and captures the cookie. Counter: FIDO2 and compliant-device policy.

Infostealers

Malware exports cookies and tokens from browser stores and sells them as logs. Counter: EDR, device-bound sessions, short lifetimes.

Refresh token abuse

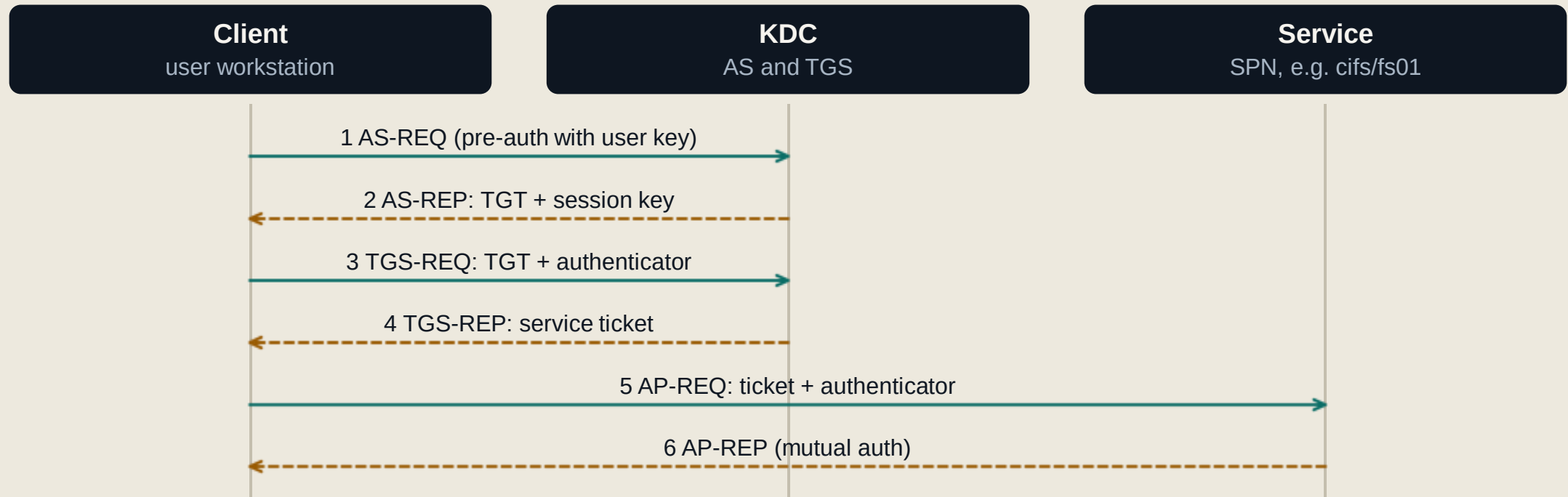
Long-lived refresh tokens mint access tokens silently. Counter: rotation with reuse detection, sender binding.

Continuous evaluation

CAE and CAEP push events (user disabled, location change) so access ends in minutes, not at token expiry.

Key idea Reduce the value of a stolen token: bind it to a device or key, shorten its lifetime, and revoke on signal.

Kerberos ticket flow



Attack surface Kerberoasting cracks service tickets offline. Golden tickets forge TGTs with the KRBTGT key. Silver tickets forge service tickets. Counter: gMSA, AES-only, KRBTGT rotation, monitor event 4769.

NTLM: understand it, then remove it

Why it lingers

Fallback when Kerberos fails: IP-address access, workgroup hosts, old apps and appliances.

What attackers do

Relay NetNTLM to LDAP, SMB or ADCS. Pass-the-hash with NT hashes. Coerce authentication. Crack captured NetNTLMv2 offline.

Removal path

- Audit usage with NTLM logging
- Enforce SMB and LDAP signing, channel binding, EPA
- Add privileged users to Protected Users
- Restrict, then disable by scope

Plan for it Microsoft has signalled NTLM deprecation. Build the inventory of NTLM dependencies now so removal is a project, not an outage.

Account recovery is where strong MFA is bypassed

The problem

Attackers skip FIDO2 by abusing MFA reset, password reset and help-desk calls. Recovery must be as strong as the strongest authenticator it can replace.

Design

Enrol two passkeys. Keep recovery codes offline. Delay and notify on recovery. Use short-lived Temporary Access Pass, then force re-enrolment.

Help-desk controls

Verification not based on knowledge data. Call back on the registered number. Video plus ID for privileged users. Cooldown and alert on MFA reset.

Key idea Treat the reset flow as an authentication path to be threat-modelled, monitored and rate-limited like any other.

CHECKPOINT 2

Check your understanding

Question	Model answer
Why is TOTP not phishing resistant?	No origin binding. A proxy relays the code inside its validity window.
State the OWASP minimum Argon2id setting.	m = 19 MiB, t = 2, p = 1 (or 46 MiB, t = 1, p = 1).
What does the __Host- cookie prefix guarantee?	Secure attribute, Path=/ and no Domain attribute, so it is host-only.
What in WebAuthn stops a look-alike site?	The browser signs the true origin and the rpId must match.
Why does Kerberoasting work?	Any user can request tickets for an SPN; the ticket is encrypted with the service key and can be cracked offline.

PART 3

Federation and protocols

How trust is expressed between systems: assertions, tokens, provisioning and shared signals.

IN THIS PART

- SAML 2.0 flow, anatomy and pitfalls
- OAuth 2.0 grants, PKCE, scopes, refresh
- OpenID Connect and JWT validation
- Sender-constrained tokens and token exchange
- SCIM, LDAP and shared signals (CAEP)
- Protocol attacks and trust design

One authentication, many relying parties

Without federation

- Every app keeps its own user store and password
- Joiner and leaver work repeated per app
- No central MFA, policy or revocation

With federation

- The IdP authenticates once and issues signed assertions
- Apps trust the IdP key and its claims
- Central MFA, conditional access, logging and revocation

Key idea Trust is the core concept: the relying party trusts the IdP signing key and the claims it makes. Your security is the IdP's security, so treat it as Tier 0.

Which protocol for which job

Protocol	Purpose	Artifact	Typical use
SAML 2.0	Web SSO, enterprise federation	XML assertion	Enterprise SaaS, legacy apps
OAuth 2.0	Delegated authorization	Access token	API access, service-to-service
OpenID Connect	Authentication on OAuth	ID token (JWT)	Modern web, mobile, SSO
Kerberos	Ticket-based SSO in a realm	TGT, service ticket	Active Directory, on-prem
LDAP	Directory query and bind	Bind credentials	Lookups, legacy authentication
SCIM 2.0	Account provisioning	JSON resources	Joiner, mover, leaver automation
RADIUS, TACACS+	Network device AAA	Shared secret	VPN, 802.1X, device admin
SSF, CAEP	Security event sharing	SET (signed JWT)	Cross-vendor session revocation

Components of SAML

Roles

Identity provider (IdP) issues assertions. Service provider (SP) consumes them. The user agent carries messages between them.

Assertions

Authentication statement (how and when), attribute statement (groups, email), optional authorization decision.

Bindings

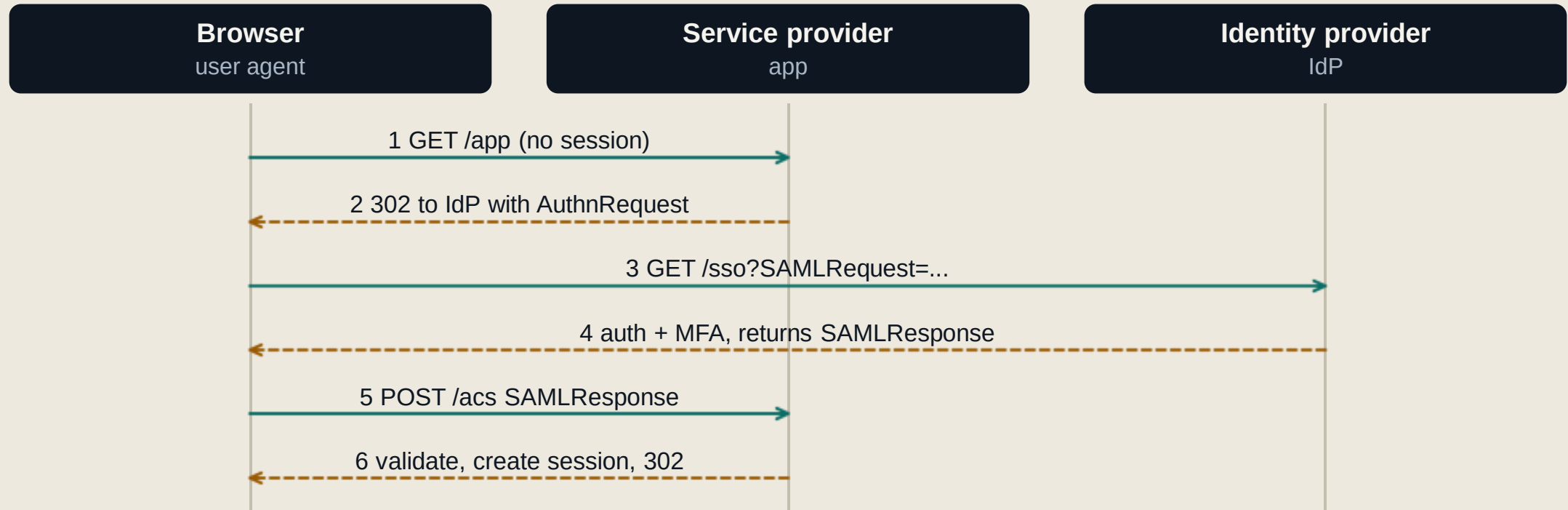
How messages travel: HTTP-Redirect (deflated, in URL), HTTP-POST (auto-submitted form), Artifact (back channel).

Metadata

entityID, ACS URL, signing and encryption certificates, NameID formats. Exchanged out of band and monitored.

Watch out SAML is XML with signatures. Most SAML vulnerabilities live in parsing and validation, not in the protocol design.

SP-initiated web browser SSO



Bearer by design Whoever presents the signed assertion at the ACS is logged in. Audience, recipient, timing and InResponseTo checks make it safe.

Anatomy of a SAML assertion

Validate every teal line

- Recipient equals your ACS URL
- InResponseTo matches your request
- NotOnOrAfter not expired, small skew
- Audience equals your entityID

Trust the right thing

- Pin the IdP certificate from metadata
- Verify the signature on the element you consume

xml

```
<saml:Assertion ID="_a75ad" IssueInstant="...">
  <saml:Issuer>https://idp.example.com</saml:Issuer>
  <ds:Signature>...</ds:Signature>
  <saml:Subject>
    <saml:NameID>alice@example.com</saml:NameID>
    <saml:SubjectConfirmationData
      Recipient="https://sp.example.com/acs"
      InResponseTo="_req1"
      NotOnOrAfter="2026-09-20T09:05:03Z"/>
    </saml:Subject>
    <saml:Conditions NotBefore="..." NotOnOrAfter="...">
      <saml:Audience>https://sp.example.com</saml:Audience>
    </saml:Conditions>
    <saml:AuthnStatement AuthnInstant="...">
      <saml:AttributeStatement> groups, dept ...
```

SAML attacks and defences

XML signature wrapping

Attacker moves the signed element and injects an unsigned one. Validate the exact element you process, by ID reference, with a maintained library.

Weak validation

Missing Audience, Recipient, NotOnOrAfter or InResponseTo checks. No replay cache of assertion IDs.

Parser attacks

XXE and DTD abuse, canonicalisation quirks such as comment injection into NameID. Disable DTDs and external entities.

Signing key compromise

A stolen IdP signing key lets an attacker forge assertions for anyone (Golden SAML). Protect and rotate the key, monitor metadata.

Key idea Prefer OIDC for new integrations. Where SAML remains, use vetted libraries and test with negative cases.

Four roles and one rule

Resource owner

The user or organisation that owns the data and grants access.

Client

The app requesting access. Public (SPA, mobile) cannot keep a secret. Confidential (server) can.

Authorization server

Authenticates the owner, obtains consent, issues tokens. Entra ID, Okta, Keycloak.

Resource server

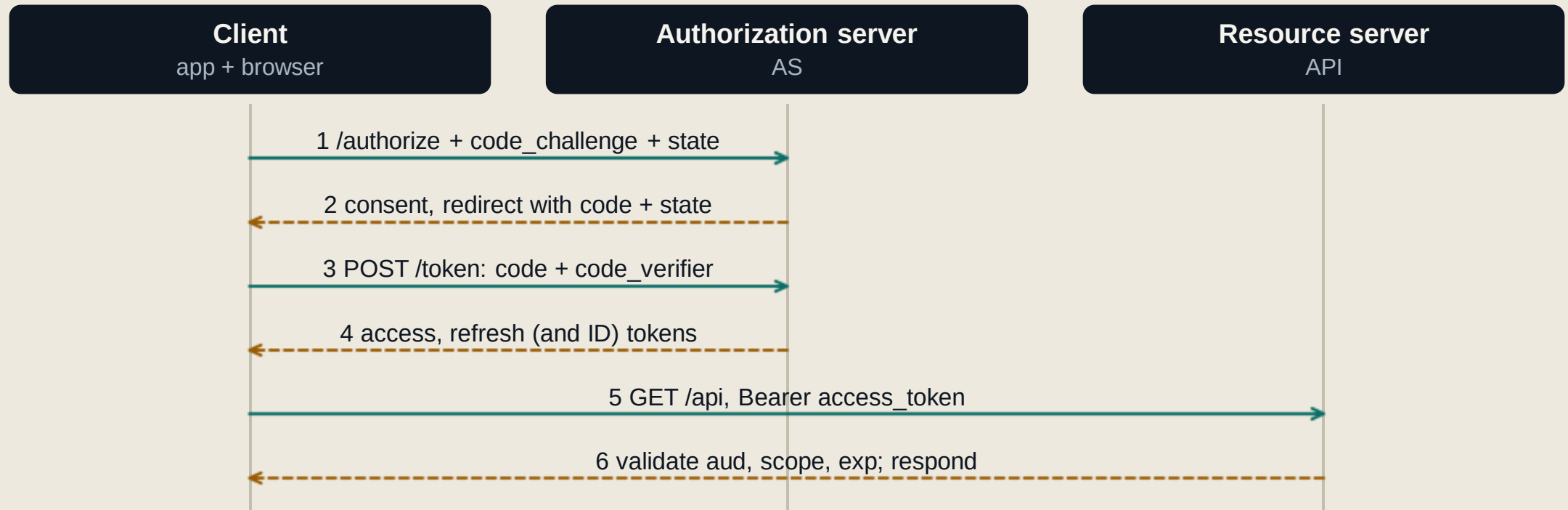
The API. Validates each token: signature or introspection, audience, scope, expiry.

Watch out OAuth is delegation, not authentication. An access token says the bearer may call an API with certain scopes. It does not say who logged in. Use OpenID Connect for sign-in.

Grant types and when to use them

Grant	Use when	Notes
Authorization code + PKCE	Web, SPA and mobile with a user	Default for every client. OAuth 2.1 requires PKCE
Client credentials	Service to service, no user	Confidential clients. Prefer private_key_jwt or mTLS
Device authorization (RFC 8628)	TVs, CLIs, IoT	User approves on another device. Target of device-code phishing
Refresh token	Renew access tokens	Rotate and sender-constrain
JWT bearer (RFC 7523)	Trust an existing assertion	Basis of workload identity federation
Token exchange (RFC 8693)	Delegation, downscoping	Microservice call chains
Implicit and password (ROPC)	Never	Removed in OAuth 2.1: tokens in URLs, passwords to clients

Authorization code flow with PKCE



Annotation The front channel carries only the code and state. Tokens travel on the back channel over TLS, after the client proves it holds the verifier.

PKCE mechanics on the wire

What it stops

- Code interception and injection
- A stolen code is useless without the verifier

Rules

- S256 only, verifier 43 to 128 chars
- Exact-match redirect_uri
- state for CSRF, nonce for OIDC

http

1. client makes a one-time secret

```
code_verifier = b64url(random(32))
```

```
code_challenge = b64url(SHA256(code_verifier))
```

2. /authorize carries only the challenge

```
GET /authorize?response_type=code
    &client_id=app1&scope=openid%20orders.read
    &redirect_uri=https://app.example.com/cb
    &state=xyz&code_challenge_method=S256
    &code_challenge=E9Melhoa2Ow...
```

3. /token proves possession

```
POST /token grant_type=authorization_code
    &code=SpIxIOBe&code_verifier=dBjftJeZ...
```

How a client authenticates to the token endpoint

Method	How it works	Assessment
client_secret_basic / post	Shared secret in header or body	Weak. Vault it, rotate it, avoid for high value
private_key_jwt	Client signs a JWT assertion with its private key	Strong. No shared secret at the AS
tls_client_auth	Mutual TLS with a client certificate	Strong. Also enables certificate-bound tokens
client_secret_jwt	HMAC-signed assertion with shared secret	Avoid. Still a shared secret
none (public client)	PKCE only	For SPAs and mobile. No secret to protect

Least privilege Service-to-service tokens should be short-lived, narrowly scoped and audience-restricted. Prefer federated workload identity to stored client secrets.

Least privilege inside the token

Scopes

Coarse, verb-oriented strings such as orders.read. Keep few and meaningful. Show them at consent. Never treat scope as a full authorization policy.

Audience and resources

The aud claim binds a token to one API. RFC 8707 resource indicators let the client ask for a specific audience, blocking cross-API replay.

Rich Authorization Requests

RFC 9396 authorization_details carries structured requests: type, actions, locations, amount. Fits payments and fine-grained data access.

Key idea A token that can only call one API, for one purpose, for five minutes, is a small blast radius even when stolen.

Refresh tokens: power that must be contained

Rotation and reuse detection

Each refresh returns a new refresh token and invalidates the old one. Reuse of an old token revokes the whole family.

Lifetimes

Access tokens in minutes. Refresh tokens with sliding and absolute limits. Request `offline_access` only when needed.

Storage and binding

Use platform secure storage or a backend-for-frontend. Bind with DPoP or mTLS. Revoke on password change or risk.

Attack path Infostealers target refresh tokens because they mint fresh access tokens silently, often bypassing MFA.

OIDC is an identity layer on OAuth 2.0

OpenID Connect 1.0

ID token, UserInfo endpoint, discovery, standard claims and scopes (openid, profile, email)

OAuth 2.0 with PKCE

Authorization code, access and refresh tokens, scopes, consent

JOSE

JWS signatures, JWE encryption, JWK keys and JWT format

HTTPS and TLS

Transport confidentiality and integrity

Discovery GET /.well-known/openid-configuration returns authorization_endpoint, token_endpoint, userinfo_endpoint, jwks_uri and supported algorithms.

Anatomy of an ID token

Stable identity key

- Use iss plus sub as the key
- Never key accounts on email

Assurance signals

- acr and amr say how the user signed in
- auth_time supports max_age checks

```
json
// header
{ "alg": "RS256", "kid": "2026-09", "typ": "JWT" }
// payload
{
  "iss": "https://login.example.com",
  "sub": "8d0f2a91",
  "aud": "app1",
  "exp": 1790000900, "iat": 1790000000,
  "nonce": "n-0S6_WzA2Mj",
  "auth_time": 1789999950,
  "acr": "urn:mfa", "amr": ["pwd", "hwk"]
}
```

Validation checklist for any JWT

1	Verify the signature with the issuer key from JWKS (by kid). Allow-list algorithms. Reject none. Never let the token choose HS versus RS.	SIGNATURE
2	iss exactly equals the configured issuer. In multi-tenant setups compare the tenant-specific issuer.	ISSUER
3	aud contains your client_id or API identifier.	AUDIENCE
4	exp is in the future, nbf in the past, clock skew at most 60 seconds.	TIME
5	nonce matches the login request (ID tokens). typ is at+jwt for access tokens (RFC 9068).	BINDING
6	Check scope, roles and acr for the operation. A valid signature is not an authorization decision.	AUTHZ

TOKENS

Opaque versus self-contained access tokens

Format	Strengths	Weaknesses	Use
Opaque + introspection (RFC 7662)	Revocable, no claim leakage, central control	A network call per validation	High-risk APIs
JWT access token (RFC 9068)	Offline validation, low latency	Revocation lag, claim bloat, data exposure	High-throughput internal APIs
Opaque at the edge, JWT inside	Combines both	Gateway complexity	Large estates

Key idea Lifetime and audience restriction matter more than format. Add revocation (RFC 7009) and introspection where the risk justifies the cost.

Sender-constrained tokens: DPoP and mTLS

DPoP (RFC 9449)

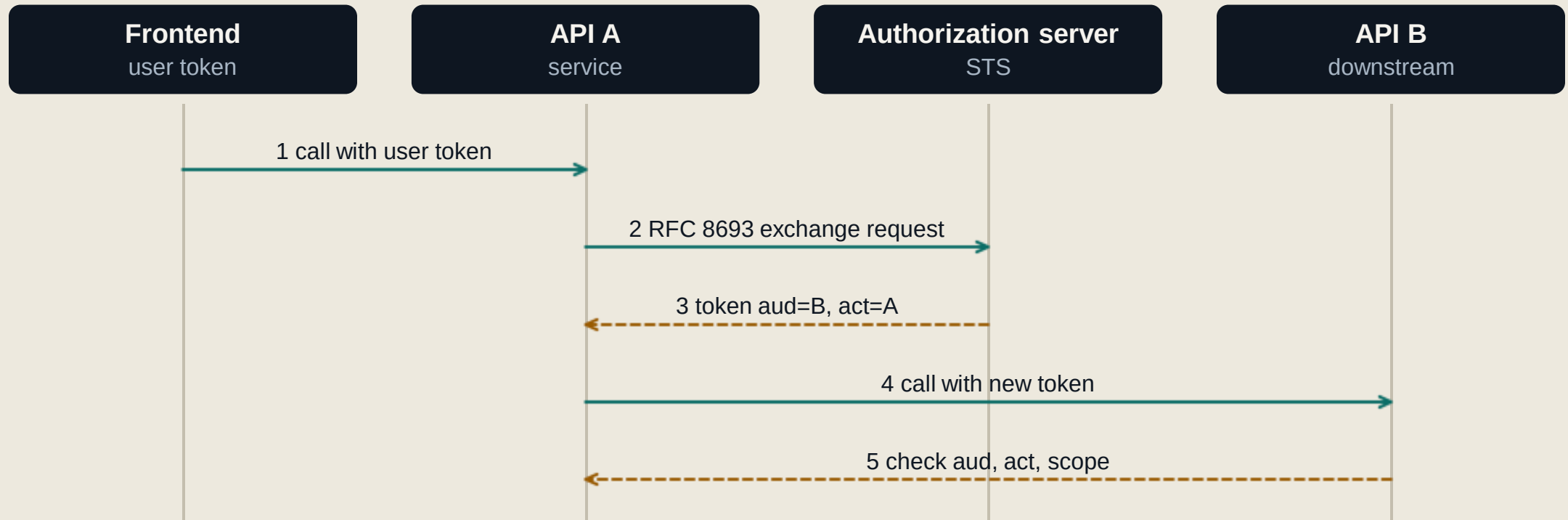
- Client holds a key pair
- Each request carries a signed proof JWT: method, URL, time, jti, token hash
- Token contains cnf.jkt, the key thumbprint

mTLS-bound (RFC 8705)

- Client authenticates with a certificate
- Token contains cnf.x5t#S256
- API checks the TLS client certificate matches

Key idea A stolen bearer token replays from anywhere. A sender-constrained token also needs the private key, which should be non-exportable (WebCrypto, TPM, secure enclave). Recommended for high-value APIs such as FAPI 2.0.

Delegation across a service chain (RFC 8693)



Key idea The act claim records who is acting for whom. Downscoping gives each hop only the authority it needs and avoids forwarding a broad user token (the confused deputy).

Provisioning with SCIM (RFC 7643, 7644)

Endpoints

- /Users, /Groups, /Bulk
- /Schemas, /ServiceProviderConfig

Design notes

- Drive from HR events
- Deprovision with active=false first
- SCIM manages accounts, not decisions

http

POST /scim/v2/Users

Content-Type: application/scim+json

```
{  
  "schemas": ["urn:ietf:params:scim:schemas:core:2.0:User"],  
  "userName": "alice@example.com",  
  "externalId": "E-104233",  
  "name": {"givenName": "Alice", "familyName": "Ng"},  
  "active": true  
}
```

Leaver

PATCH /scim/v2/Users/{id}

```
{  
  "Operations": [{"op": "replace", "path": "active", "value": false}]  
}
```

Directory protocol essentials

Model

A tree of entries. Each has a DN such as uid=alice,ou=Staff,dc=corp,dc=example, object classes and attributes defined by schema. Filters look like (&(objectClass=user)(department=Finance)).

Bind and transport

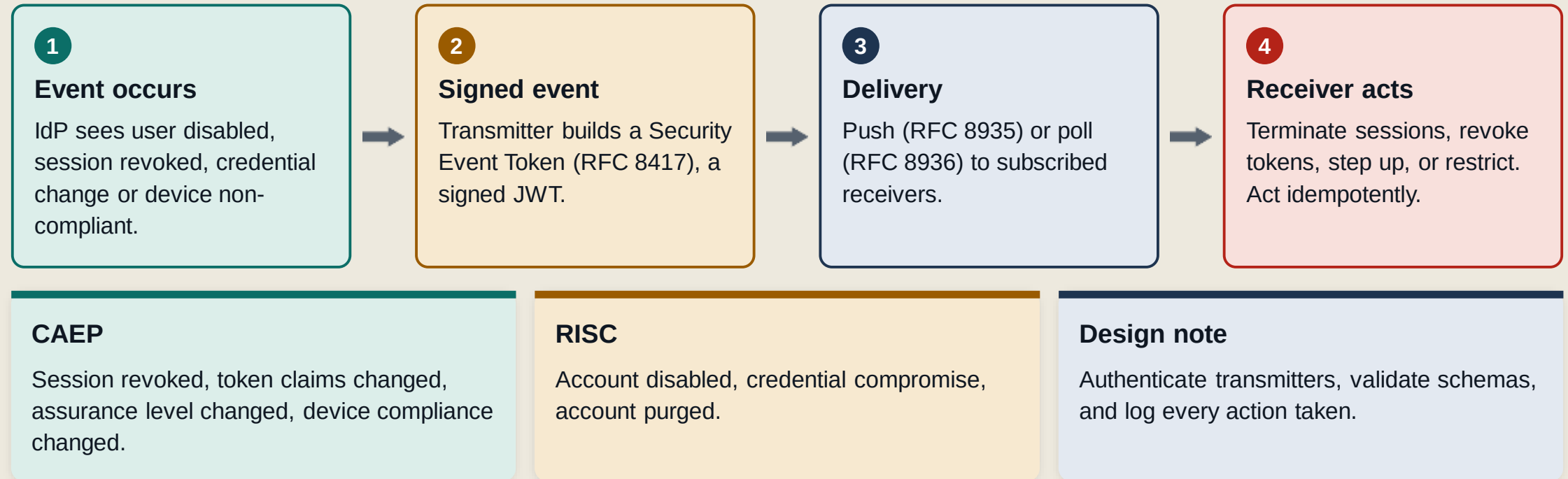
Simple bind sends credentials, so require LDAPS or StartTLS. SASL GSSAPI uses Kerberos. On AD enforce LDAP signing and channel binding. No anonymous or unsigned binds.

Pitfalls

Applications binding with over-privileged service accounts. LDAP injection through unescaped filters. Using LDAP as an authentication API where OIDC would do.

Rule of thumb Give each application a read-only bind account scoped to its own OU, or move it to OIDC or SAML.

CAEP and SSF: revoke access in near real time



OAuth and OIDC weaknesses (RFC 9700 security BCP)

Weakness	Effect	Mitigation
Loose redirect_uri or open redirect	Code or token sent to attacker	Exact-match registered URIs
Missing state or nonce	CSRF login, replayed responses	Random per request, validate on return
Code interception or injection	Attacker redeems the code	PKCE S256, one-time short-lived codes
Mix-up across multiple IdPs	Code sent to the wrong AS	Issuer identification (RFC 9207)
Illicit consent grant	Persistent API access with broad scopes	Admin consent workflow, scope review
Device-code phishing	Victim approves attacker session	Restrict flow, conditional access
Audience misuse	Token replayed at another API	aud check, resource indicators

Federation extends least privilege across boundaries

Minimise released claims

Release only the attributes each SP needs. Use pairwise subject identifiers where correlation is a risk.

Map, do not trust

Map external groups to internal roles. Deny unmapped values. Never let an external claim grant admin.

Key on issuer plus subject

With several IdPs, identity is (iss, sub). Email can collide or be re-assigned.

Govern B2B guests

Sponsor, expiry, access review, and clear rules on which tenant enforces MFA and device policy.

Key idea The more claims you accept and release, the wider the trust surface. Federation without attribute discipline is privilege delegation without review.

CHECKPOINT 3

Check your understanding

Question	Model answer
Why must an SP check Audience and InResponseTo?	So assertions for another SP, or unsolicited or replayed ones, are rejected.
What does PKCE protect against?	Interception or injection of the code. It is useless without the verifier.
Which two claims form a stable OIDC user key?	iss and sub, never email.
Give one benefit of sender-constrained tokens.	A stolen token cannot be replayed without the private key or certificate.
Why is the implicit grant deprecated?	Tokens leak through URLs, history and referrers, with no sender constraint. Code plus PKCE replaces it.

PART 4

Authorization models and policy

How systems decide what an authenticated principal may do, and how to express that as testable policy.

IN THIS PART

- Where authorization fails
- ACL, DAC, MAC, RBAC, ABAC, ReBAC
- Policy engines: OPA, Cedar, OpenFGA
- Enforcement patterns from gateway to database
- Separation of duties and testing policy

Where authorization fails

Horizontal (BOLA, IDOR)

User A reads user B's object by changing an ID. The check on ownership is missing.

Vertical (BFLA)

A normal user calls an admin function because only the UI hid the button.

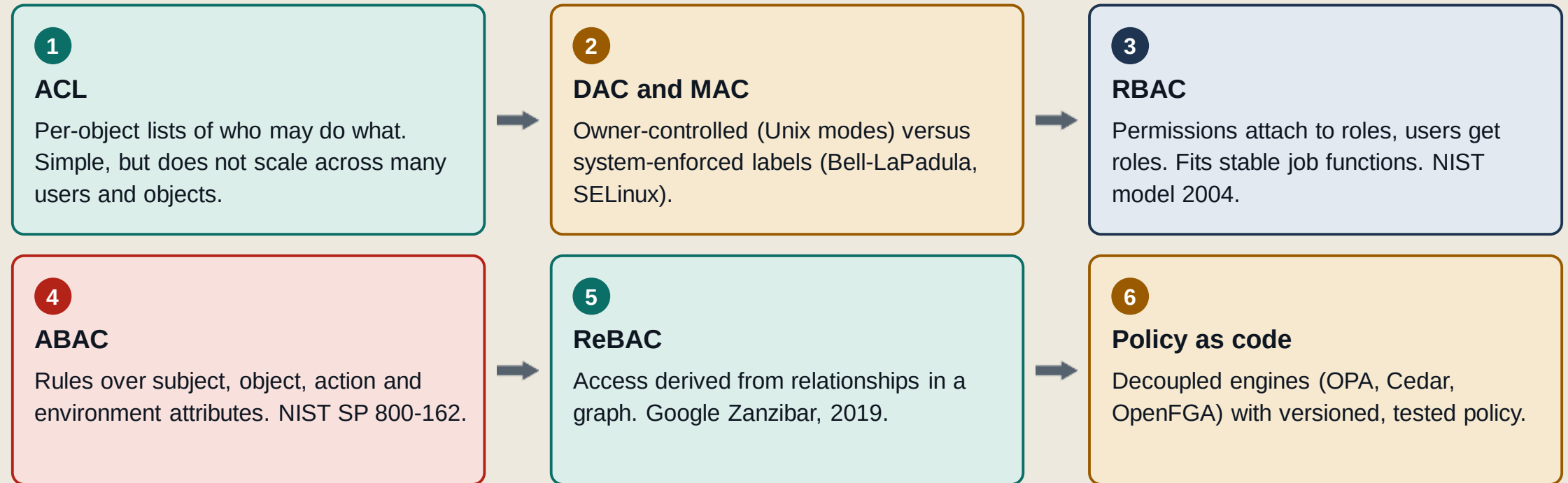
Data and context

Mass assignment sets role=admin. Excessive fields returned. Tenant ID trusted from the request body.

Key idea Broken access control tops the OWASP Top 10 and BOLA tops the API Security Top 10. Authorization is the most common application security failure.

EVOLUTION

Six generations of access control



Key idea Models are not replacements. Real systems layer them: mandatory guardrails, RBAC for job function, ABAC for context, ReBAC for object sharing.

ACLs, DAC and MAC

ACLs and file modes

Unix `rw-r-x---` and POSIX ACLs, NTFS access control entries with inheritance. Ownership implies control. Good for files, poor for enterprise-wide review.

DAC versus MAC

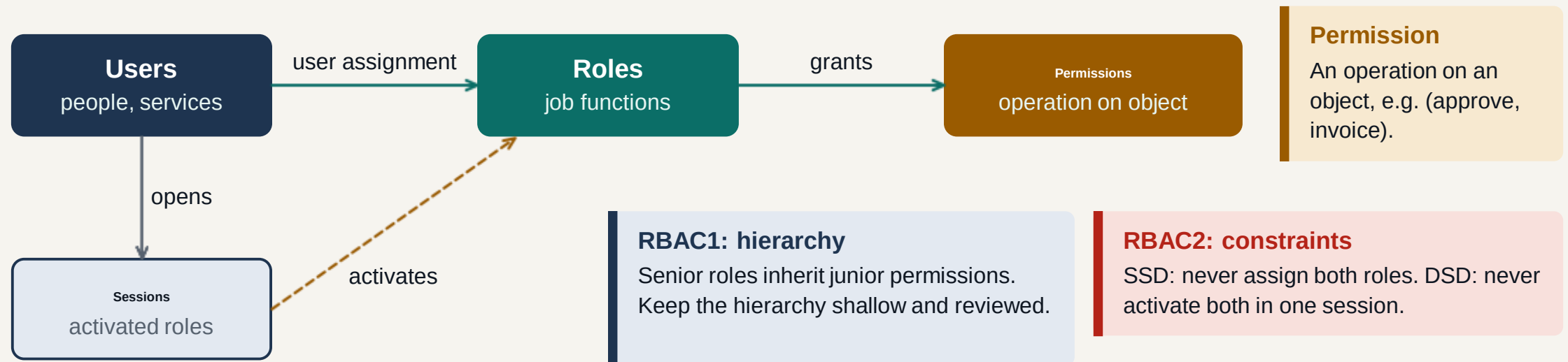
DAC: owners grant access at their discretion. MAC: the system enforces labels and clearances. Bell-LaPadula protects confidentiality (no read up, no write down). Biba protects integrity.

Mandatory in practice

SELinux and AppArmor confine processes. Cloud SCPs and org policies act as mandatory guardrails over grants. Sensitivity labels drive DLP.

Watch out DAC lets any owner over-share. Without a mandatory layer, one careless grant defeats the whole policy.

The NIST RBAC model, RBAC0 to RBAC3



Key idea RBAC0 is users, roles, permissions and sessions. RBAC1 adds hierarchy. RBAC2 adds constraints. RBAC3 combines both.

Three ways RBAC decays

Role explosion

A role per app, region, project and level multiplies into hundreds. Fix: keep roles for job function and parameterise scope with attributes.

Role creep

Roles quietly gain permissions over time. Fix: recertify role contents, track permission usage, shrink.

Catch-all roles

Admin, PowerUser and Contributor cover too much. Fix: split by task, add JIT elevation, use custom roles.

Metric Track roles per user, permissions per role and the share of permissions never used. Rising numbers signal design debt.

Attribute-based access control (NIST SP 800-162)

Strengths

- Fine-grained and contextual
- Fewer roles, scales with attributes

Costs

- Attribute quality and governance
- Hard to answer who can access X
- Policy testing is harder

pseudo-policy

attributes

subject: dept=risk, clearance=secret

object: class=secret, owner_dept=risk

action: read

environment: device=managed, net=corp

rule

permit if

subject.clearance >= object.class

and subject.dept == object.owner_dept

and environment.device == "managed"

Watch out ABAC is only as good as its attributes. Stale or self-asserted attributes produce wrong decisions silently.

Relationship-based access and Google Zanzibar

Model

Tuples form a graph. A namespace config says how relations compose, e.g. viewer includes editor and parent viewers.

Consistency

Consistency tokens avoid the new enemy problem: a stale ACL applied to fresh content.

Fits

Sharing, folders, orgs and teams in multi-tenant SaaS.

zanzibar tuples

```
# object#relation@subject
```

```
doc:plan#owner@user:alice
```

```
doc:plan#parent@folder:q3
```

```
folder:q3#viewer@group:finance#member
```

```
group:finance#member@user:bob
```

```
# Check
```

```
check(doc:plan#viewer@user:bob)
```

```
-> viewer includes parent folder viewers
```

```
-> folder:q3 viewer includes finance members
```

```
-> bob is a member => ALLOW
```

COMPARISON

Choosing an authorization model

Model	Best for	Strength	Weakness
RBAC	Stable job functions, audits	Simple, reviewable, well understood	Role explosion, no context
ABAC	Contextual, data-centric rules	Expressive, few roles	Attribute governance, opaque reviews
ReBAC	Object sharing, hierarchies	Natural for ownership and teams	Graph size, consistency, tooling
Policy-based hybrid	Enterprise platforms	Guardrails plus grants plus context	Needs a policy pipeline

Key idea Most estates combine them: RBAC for coarse job function, ABAC for scope and context, ReBAC for object sharing.

ENFORCEMENT

Where to place the PEP and PDP

Pattern	How it works	Strength	Trade-off
Embedded library	Engine runs in-process	Lowest latency	Per-language, redeploy to update
Sidecar or agent	OPA or Envoy ext_authz next to the service	Low latency, central bundles	Operational overhead
Central PDP service	Network call to a decision API	One source of truth	Latency and availability
Gateway PEP	Checks at the API gateway	Uniform edge control	Coarse-grained only
Database layer	Row-level security in the store	Strongest safety net	Harder to express business rules

Key idea Enforce in layers: gateway for coarse checks, service for object-level checks, database as the last line.

Open Policy Agent and Rego

Why OPA

- Decoupled from application code
- Default deny, data plus input
- Decision logging and bundles

Tips

- Keep policy small and readable
- No secrets in policy
- Write negative tests

rego

```
package api.authz
```

```
default allow := false
```

```
allow if {
```

```
  input.method == "GET"
```

```
  input.path == ["orders", order_id]
```

```
  order := data.orders[order_id]
```

```
  order.tenant == input.user.tenant
```

```
  "orders.read" in input.token.scope
```

```
}
```

```
allow if { "admin" in input.user.roles }
```

Cedar: readable, analyzable policy

Semantics

- Default deny
- Any matching forbid overrides permit
- Order-independent, schema-validated

Used in

- Amazon Verified Permissions
- Applications that need analyzable policy

```
cedar
permit (
  principal in Group::"finance",
  action in [Action::"read", Action::"list"],
  resource in Folder::"q3-reports"
)
when { context.mfa == true };

forbid (
  principal,
  action == Action::"delete",
  resource
)
unless { resource.owner == principal };
```

OpenFGA model (Zanzibar-style DSL)

Reading the model

- editor includes owner
- viewer includes editor and parent viewers
- Inheritance is transitive

Queries

- Check(user, relation, object)
- ListObjects(user, viewer, document)
- ListUsers(document, viewer)

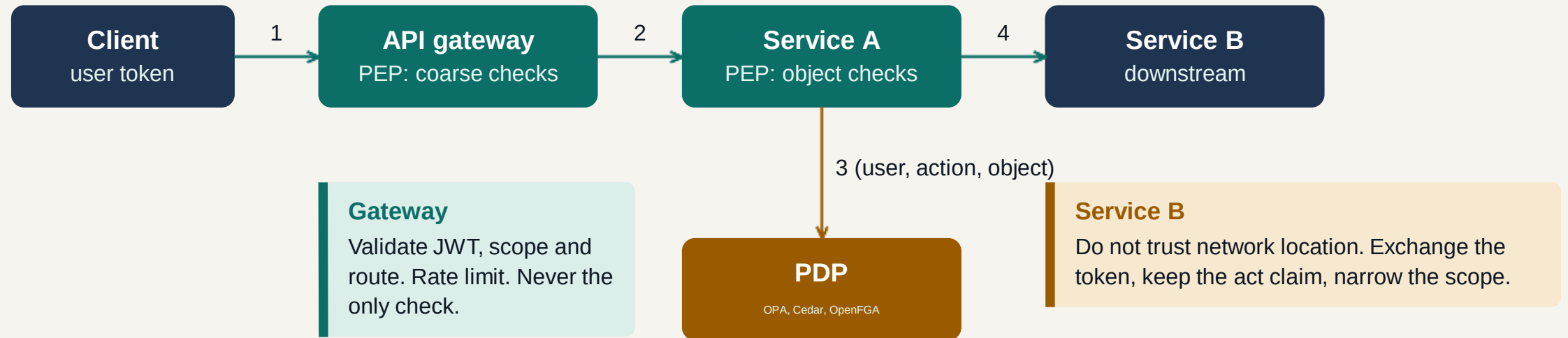
```
openfga dsl
model
  schema 1.1
type user
type group
relations
  define member: [user]
type folder
relations
  define viewer: [user, group#member]
type document
relations
  define parent: [folder]
  define owner: [user]
  define editor: [user] or owner
  define viewer: [user] or editor or viewer from parent
```

Scope, role, permission and claim are different things

Concept	Belongs to	Example	Who decides
Scope	Client delegation	orders.read	Authorization server at consent
Role	User grouping	OrdersClerk	IAM administrator
Permission	Resource server	orders:approve	Resource server policy
Claim	Token attribute	dept=risk	IdP or attribute source

Key idea Effective access is the intersection: scope narrows what the client may do, it never widens beyond the user's permissions or the resource policy.

Authorization across a service chain



Confused deputy Forwarding the user's broad token lets any service act with all of it. Exchange for a narrow, audience-bound token at each hop.

Row-level security as a safety net

Why

- Catches a missing filter in app code
- Applies to every query path, BI tools too

Also use

- Column masking and tag-based access
- Separate owner and app roles: owners bypass RLS

postgresql

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation ON orders  
USING (tenant_id =  
        current_setting('app.tenant')::uuid);
```

```
CREATE POLICY analyst_read ON orders  
FOR SELECT TO analyst_role  
USING (region = current_setting('app.region'));
```

```
-- per request, inside the transaction  
SET LOCAL app.tenant = '6f1c9d2e-...';
```

API authorization risks and how to test them

Risk	Example	Prevention and test
BOLA (API1:2023)	GET /orders/1002 returns another user's order	Object-level check on every ID. Test with two users
BFLA	A normal user calls POST /admin/users	Function-level policy, deny by default, role tests
Mass assignment	PATCH sets "role":"admin"	Allow-list writable fields, separate DTOs
Excessive data exposure	API returns full record, UI hides fields	Shape responses, field-level checks
Tenant confusion	Tenant ID taken from the request body	Derive tenant from the token or session only

SEPARATION OF DUTIES

Toxic combinations to prevent

Duty A	Duty B	Risk	Control
Create vendor	Approve payment	Fictitious vendor fraud	SSD rule, workflow block
Write code	Approve production deploy	Unreviewed change to production	Branch protection, two-person rule
Request access	Approve access	Self-approval	Block self-approval in IGA
Cloud IAM admin	Delete audit logs	Cover tracks	SCP protects logging, separate identities
Security admin	Auditor	Tampering with evidence	Separate people and tenants

Key idea Static SoD blocks assignment. Dynamic SoD requires a second approver at the moment of action.

Test authorization like code

What to test

- Each role by action by resource class
- Negative cases: cross-tenant, expired, wrong audience
- A regression test for every incident

Pipeline

- Policy in Git with PR review
- opa test in CI, diff impact analysis
- Shadow mode using decision logs

rego test

```
import data.api.authz
```

```
test_same_tenant_allowed if {  
  authz.allow with input as req("t1")  
  with data.orders as {"o1": {"tenant": "t1"}}  
}
```

```
test_cross_tenant_denied if {  
  not authz.allow with input as req("t2")  
  with data.orders as {"o1": {"tenant": "t1"}}  
}
```

```
req(t) := {"method": "GET", "path": ["orders", "o1"],  
  "user": {"tenant": t}, "token": {"scope": ["orders.read"]}}
```

CHECKPOINT 4

Check your understanding

Question	Model answer
BOLA versus BFLA?	BOLA: object-level, another user's object. BFLA: function-level, a privileged function.
What does RBAC2 add?	Constraints such as static and dynamic separation of duty.
When is ReBAC a better fit than RBAC?	Object sharing and hierarchies: owners, teams, folders, per-object access.
Why enforce at several layers?	A missed check at one layer is caught by another. The database is the last line.
What is the effective access when a client presents a scope?	The intersection of scope, the user's permissions and the resource policy.

PART 5

Least privilege in depth

The core discipline: give every identity only what it needs, only where, only when, and prove it continuously.

IN THIS PART

- Definition, origins and NIST AC-6
- Blast radius and the permission gap
- Right-sizing from real usage
- Just-in-time access and zero standing privilege
- Metrics, anti-patterns and guardrails

PART 5 THESIS

Least privilege is a design constraint, not a clean-up project.

Retrofitting it onto a mature estate costs ten times more than building it in. The goal is to make the least-privileged path the easiest path.

DEFINITION

What least privilege actually means

The principle

Saltzer and Schroeder (1975): every program and user should operate with the smallest set of privileges needed to do the job. It limits damage from error, misuse and compromise.

The control

NIST SP 800-53 AC-6 requires least privilege and lists ten enhancements: authorise security functions, use non-privileged accounts for routine work, log privileged actions, review privileges.

The four questions

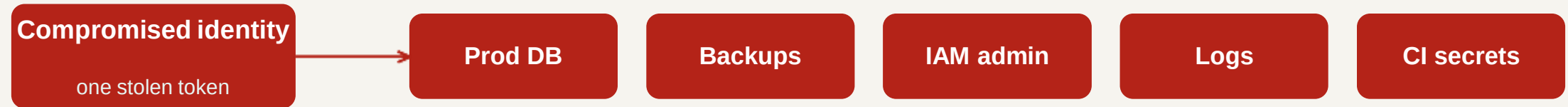
What actions? **Which** resources? **When** and for how long? **How** (device, network, MFA strength)? Least privilege constrains all four, not only the first.

Key idea Least privilege is also about time and context. A perfectly scoped role held permanently from an unmanaged laptop is still over-privileged.

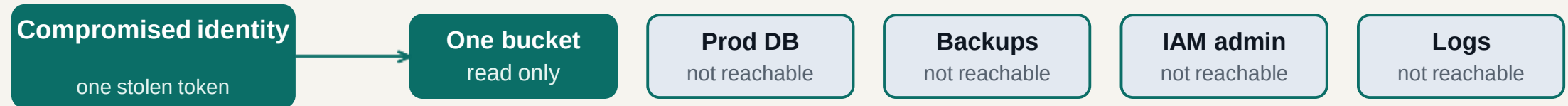
BLAST RADIUS

Blast radius: the same breach, two outcomes

BROAD ACCESS



LEAST PRIVILEGE



Blast radius = what an attacker can reach with one identity

Reduce it with narrow scope, short lifetime and segmented tiers. Detection then has time to work.

Why it matters

Ransomware and cloud breaches nearly always escalate from one over-privileged identity. Least privilege turns a breach into an incident.

DIMENSIONS

Four dimensions of a privilege

Action

Verbs granted: read, write, delete, iam:PassRole. Prefer named actions over wildcards. Separate read from write from admin.

Resource

Which objects: ARN, table, namespace, tenant. Scope by tag, path or resource ID, never Resource: "*" for anything sensitive.

Time

Standing versus temporary. Session length, expiry, JIT windows, business-hours conditions. Shorter is safer.

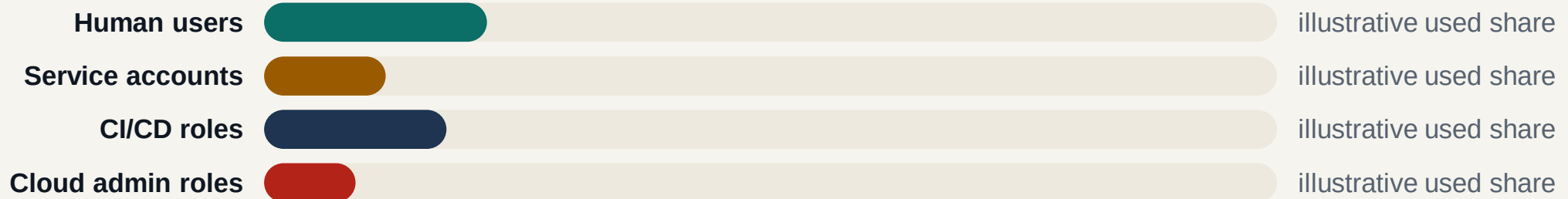
Context

Device posture, network, MFA strength, workload identity, source account. Conditions bind a grant to where it is legitimate.

Key idea Score each grant on all four axes. Over-privilege on any one of them is a finding.

PERMISSION GAP

The permission gap: granted versus used



How to measure

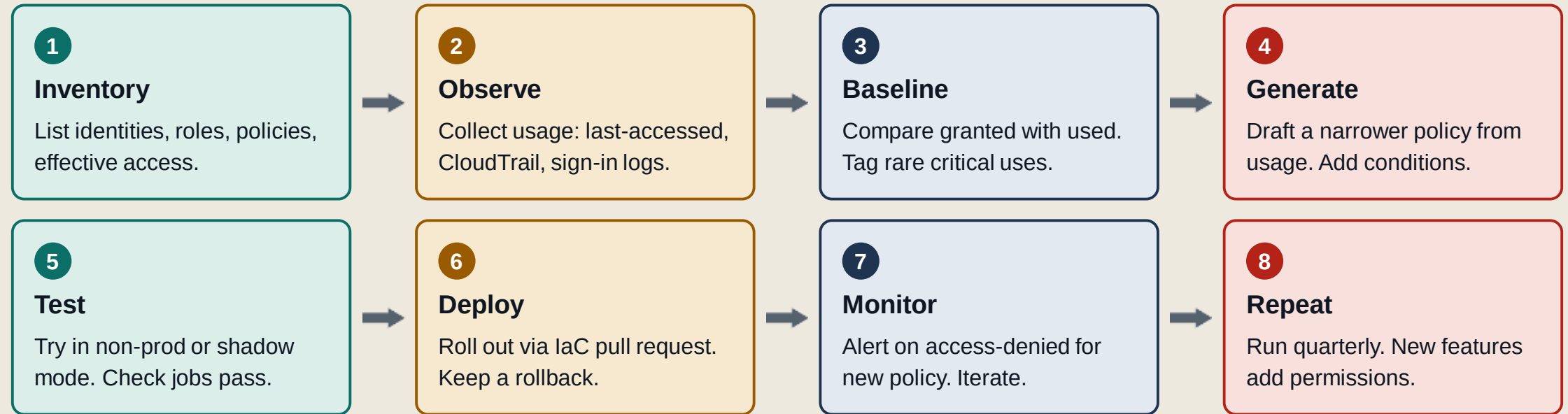
- Granted: policies plus group closure
- Used: CloudTrail, Activity Log, Audit Logs
- Gap = granted minus used, 90 days

Read it carefully

- Unused is not unnecessary: break-glass, quarterly jobs
- Use 90 to 180 day windows, tag rare-critical grants
- Bars are illustrative; measure your own

Reality Studies of cloud estates repeatedly find that only a small fraction of granted permissions are ever used. That gap is your attack surface.

The right-sizing loop



Watch out Right-size in small batches, owner by owner. Big-bang tightening breaks jobs and forces rollback.

Right-sizing with AWS tooling

Last accessed

Service and action last-accessed data shows which granted services were never used. Data covers roughly the last 400 days.

Policy generation

IAM Access Analyzer builds a policy from CloudTrail activity for a role, up to 90 days, as a starting draft.

Then

Review by an owner, add conditions, deploy through IaC.

```
aws cli
```

```
# which services has this role actually used?
```

```
aws iam generate-service-last-accessed-details \  
--arn arn:aws:iam::111122223333:role/etl-prod
```

```
aws iam get-service-last-accessed-details \  
--job-id $JOB_ID \  
--query 'ServicesLastAccessed[?!LastAuthenticated]'
```

```
# draft a policy from CloudTrail usage (90 days)
```

```
aws accessanalyzer start-policy-generation \  
--policy-generation-details \  
principalArn=arn:aws:iam::111122223333:role/etl-prod \  
--cloud-trail-details file://trail.json
```

Just-in-time access and zero standing privilege

STANDING PRIVILEGE

Admin rights held 24 x 7 for 365 days
exposed to phishing, malware, insider misuse, token theft

JUST-IN-TIME (ZSP)

Zero standing access. Eligible only.

Elevated
60 minutes

incident 1

Elevated
60 minutes

change window

Elevated
60 minutes

incident 2

1 Request

Reason, ticket, duration, resource.

2 Approve

Auto for low risk, human for high.

3 Elevate and record

MFA, session logged, auto-expire.

Key idea Exposure time falls from 8,760 hours a year to a few hours. Attackers lose the standing target.

JIT implementations across platforms

Platform	Mechanism	Typical control
Entra ID / Azure	Privileged Identity Management (PIM) eligible assignments	Max activation duration, MFA, approval, justification
AWS	IAM Identity Center permission sets, temporary elevation tooling, STS session duration	Short sessions, ticket-linked approval
GCP	IAM Conditions with expiry, Privileged Access Manager (PAM)	Time-bound grants with approvers
Kubernetes	Short-lived RoleBindings, impersonation via broker	Auto-revoke, audit of exec and secrets
Databases and servers	PAM broker issues ephemeral credentials	Session recording, command filtering

NIST SP 800-53 AC-6 enhancements as a checklist

Control	Requirement	Practical evidence
AC-6(1)	Authorise access to security functions	Role catalogue, approval records
AC-6(2)	Non-privileged accounts for non-security tasks	Separate admin accounts, PAWs
AC-6(5)	Restrict privileged accounts	Privileged inventory, tiering
AC-6(7)	Review privileges periodically	Recertification campaigns
AC-6(9)	Log use of privileged functions	Privileged action logs in the SIEM
AC-6(10)	Block non-privileged users from privileged functions	Deny tests, guardrail policy

Preventive guardrails that enforce the ceiling

Boundaries and SCPs

AWS permissions boundaries and Organizations SCPs cap what any grant can allow. Azure Policy and management-group deny. GCP organisation policies.

Conditions

Bind grants to aws:SourceVpce, aws:PrincipalOrgID, MFA age, device compliance, time and tags. A stolen key then fails outside its context.

Templates

Golden roles in IaC modules. Teams pick from a catalogue. Wildcard actions fail a policy linter in CI.

Key idea Guardrails are ceilings set by security. Grants are floors set by owners. Effective access is the intersection.

METRICS

Least privilege KPIs that mean something

Metric	Definition	Direction
Unused permission ratio	Granted minus used over 90 days, per identity class	Down
Standing privileged accounts	Count of always-active admin assignments	Toward zero
Median elevation duration	Time between JIT activation and expiry	Short, stable
Wildcard grants	Policies with Action or Resource "*" on sensitive services	Down
Stale identities	No sign-in or token use in 90 days but still enabled	Down
Time to revoke	Leaver event to all access removed	Hours, not days

Watch out Do not reward teams for fewer permissions alone. Pair reduction with incident-free operations so people do not over-restrict and cause outages.

Least privilege anti-patterns

Anti-pattern	Why it happens	Fix
Admin because it works	Debugging denied errors is slow	Access Analyzer, policy simulator, deny logs
Shared break-glass reused daily	Convenience	Vault, alert on every use, rotate after use
Copy an old user's access	Onboarding by cloning	Role-based birthright plus request catalogue
Wildcard resources on PassRole	Hard to enumerate roles	Restrict roles and add PassedToService condition
Exceptions with no expiry	No owner or end date	Every exception carries an expiry and owner

CHECKPOINT 5

Check your understanding

Question	Model answer
Name the four dimensions of a privilege.	Action, resource, time and context.
What is zero standing privilege?	No permanent elevated access. Identities are eligible and elevate just in time, time-boxed and logged.
Why is unused not always unnecessary?	Rare but critical duties such as disaster recovery or quarterly jobs may not appear in a short window.
What sets the ceiling on effective permissions?	Guardrails: boundaries, SCPs and organisation policies intersect with grants.
Why right-size in small batches?	It limits outages and preserves trust so the programme is not rolled back.

PART 6

Privileged access and secrets

Controlling the identities and credentials that can change everything: vaulting, session control, dynamic secrets and workload identity.

IN THIS PART

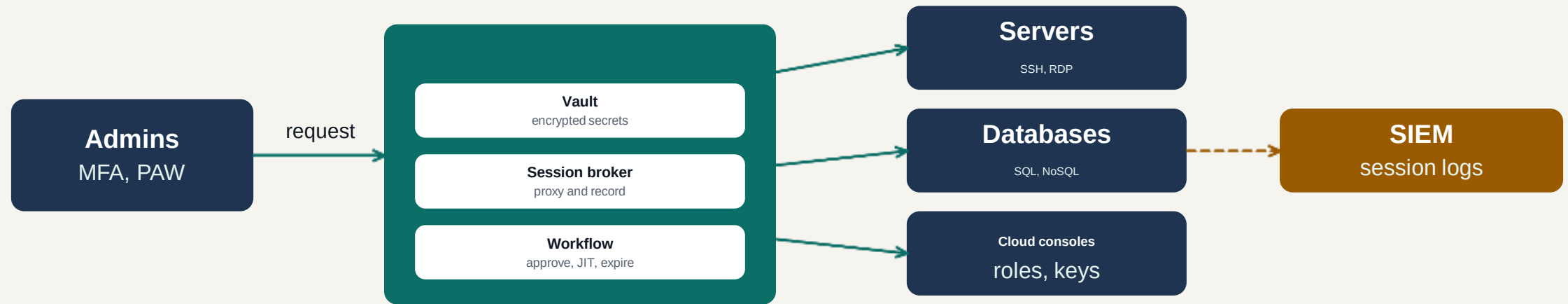
- Privileged account taxonomy
- PAM architecture, vaulting and session control
- LAPS and local admin
- Secrets sprawl and dynamic secrets
- SPIFFE and secretless workloads
- Break-glass design

Privileged account types and their risks

Type	Example	Main risk	Preferred control
Personal admin	alice-adm, Domain Admin	Phishing of a daily-use account	Separate account, PAW, JIT
Shared and local admin	Administrator, root	Reuse and lateral movement	LAPS, vault, unique per host
Service account	svc-sql running a service	Never rotated, over-scoped	gMSA, managed identity, rotate
Application secrets	API keys, DB passwords	Leaks in code and logs	Vault, dynamic secrets
Emergency	Break-glass, firecall	Silent misuse	Sealed, alarmed, tested
Cloud root and owner	AWS root, Global Admin	Total tenant loss	Hardware MFA, no daily use

Key idea Inventory first. Most breaches start with a privileged account the security team did not know existed.

Reference architecture for privileged access management



Credential never seen by the admin

The broker injects the secret into the session.
The human gets access, not the password.

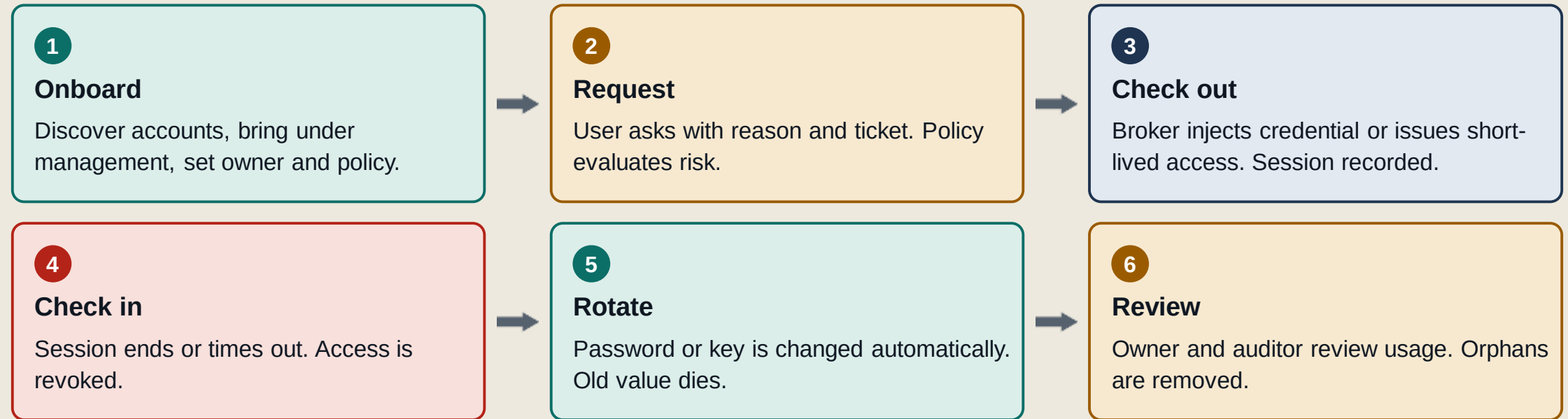
Every session recorded

Keystroke and video logs, command filtering
and live termination for high risk.

Rotate after use

Checked-in secrets are rotated. A leaked value
is worthless within minutes.

Vaulted credential lifecycle



Watch out Rotation must also update dependants (services, jobs, connection strings). Map dependencies before you rotate a service account.

Windows LAPS: unique local admin per machine

Problem

A shared local admin password lets one compromised host unlock the fleet through pass-the-hash.

Solution

Windows LAPS rotates a unique password per device and stores it in AD or Entra ID, encrypted.

Access

Only named groups may read it, and reads are audited.

powershell

```
# extend AD schema and grant self-write on the OU
```

```
Update-LapsADSchema
```

```
Set-LapsADComputerSelfPermission `
```

```
-Identity "OU=Servers,DC=corp,DC=example"
```

```
# allow a helpdesk group to read passwords
```

```
Set-LapsADReadPasswordPermission `
```

```
-Identity "OU=Servers,DC=corp,DC=example" `
```

```
-AllowedPrincipals "CORP\LAPS-Readers"
```

```
# retrieve (audited)
```

```
Get-LapsADPassword -Identity SRV01 -AsPlainText
```

Where secrets leak

Location	Typical cause	Control
Git history	Committed key, .env file	Pre-commit and server-side secret scanning, rotate on leak
CI/CD logs and variables	Echoed variables, broad pipeline access	Masked secrets, OIDC federation, scoped runners
Container images	Baked credentials in layers	Build-time secrets, image scanning
Chat and tickets	Pasted for convenience	DLP, share via vault link
Config and env vars	Long-lived static values	Dynamic secrets, secret manager references
Developer laptops	~/.aws, kubeconfig, tokens	Short-lived SSO sessions, device posture

Reality Treat any committed secret as compromised, even in a private repo. Rotate first, then clean history.

Dynamic secrets: credentials created on demand

Idea

The secrets manager creates a unique database user per request with a short TTL, then drops it.

Benefits

No shared static password, automatic revoke, per-consumer audit trail.

Requires

A privileged connection for the manager and an app that can renew or refetch.

hashicorp vault

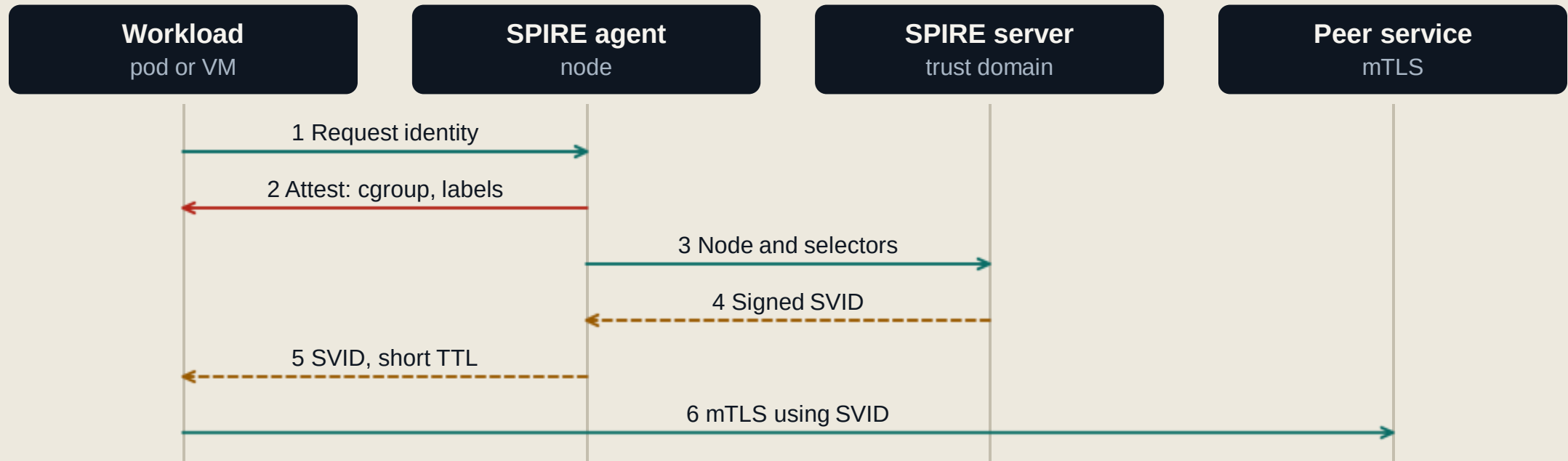
```
# configure a database secrets engine
vault write database/config/orders \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@db/orders" \
  allowed_roles="readonly"

# a role that creates a read-only user for 1 hour
vault write database/roles/readonly \
  db_name=orders default_ttl=1h max_ttl=4h \
  creation_statements=@create_ro.sql

# create_ro.sql (template)
CREATE ROLE "{{name}}" LOGIN PASSWORD '{{password}}'
  VALID UNTIL '{{expiration}}';
GRANT SELECT ON ALL TABLES IN SCHEMA public
  TO "{{name}}";

vault read database/creds/readonly
```

SPIFFE and SPIRE: workload identity without secrets



Key idea The identity is a URI, `spiffe://corp.example/ns/prod/sa/orders`, proven by attestation, not a stored password.

Ranking credential strategies for workloads

Strategy	Secret at rest?	Lifetime	Use when
Static API key	Yes	Months to forever	Avoid; last resort with rotation
Secrets manager, rotated	Yes, managed	Days to weeks	Third-party APIs with no federation
Vault dynamic secrets	No long-lived	Minutes to hours	Databases, cloud creds
Managed identity / IAM role	None	Token, about an hour	Workloads on the same cloud
OIDC federation (WIF)	None	Token, minutes	CI/CD and cross-cloud
SPIFFE SVID and mTLS	None	Minutes to hours	Service-to-service, multi-cloud

Key idea Move down this list over time. The best secret is one that does not exist.

Designing break-glass accounts

1	Two or more cloud-only emergency accounts, excluded from Conditional Access outages	SCOPE
2	Long random credentials split between two custodians, hardware FIDO2 key stored sealed	SECRET
3	Excluded from MFA that depends on the failing system, but never from logging	EXCEPTION
4	Alert to the SOC on any sign-in. Auto-open an incident	ALARM
5	Test quarterly, rotate after every use, document the procedure	TEST

Watch out Microsoft advises excluding at least one emergency account from Conditional Access policies that could lock everyone out, with compensating monitoring.

REMEDIATION

When a secret leaks: first hour

Contain

Revoke or rotate the credential immediately. Do not wait for scope analysis.

Assess

Check audit logs for use of the key since exposure. Identify what it could reach.

Clean and learn

Remove from history where feasible, add a scanner rule, fix the root cause such as a shared example file.

Metric Track mean time to rotate a leaked secret. Hours is achievable when rotation is automated.

CHECKPOINT 6

Check your understanding

Question	Model answer
Why should admins not know the target password?	Injection prevents theft, reuse and sharing. Access is granted without disclosure.
What problem does LAPS solve?	Shared local admin passwords that enable lateral movement by pass-the-hash.
Benefit of dynamic secrets?	Unique, short-lived credentials with automatic revocation and per-consumer audit.
How does SPIFFE prove a workload identity?	Attestation of the node and workload, then a short-lived signed SVID.
Why alert on every break-glass use?	It is rare by design, so any use is either an emergency or an attack.

PART 7

Cloud IAM across AWS, Azure, GCP and Kubernetes

Policy evaluation, guardrails, workload federation and the escalation paths that turn small permissions into full control.

IN THIS PART

- Model comparison across clouds
- AWS policy evaluation, SCPs and OIDC federation
- Azure RBAC, PIM and scope
- GCP hierarchy, conditions and impersonation
- Kubernetes RBAC
- Privilege escalation paths and CIEM

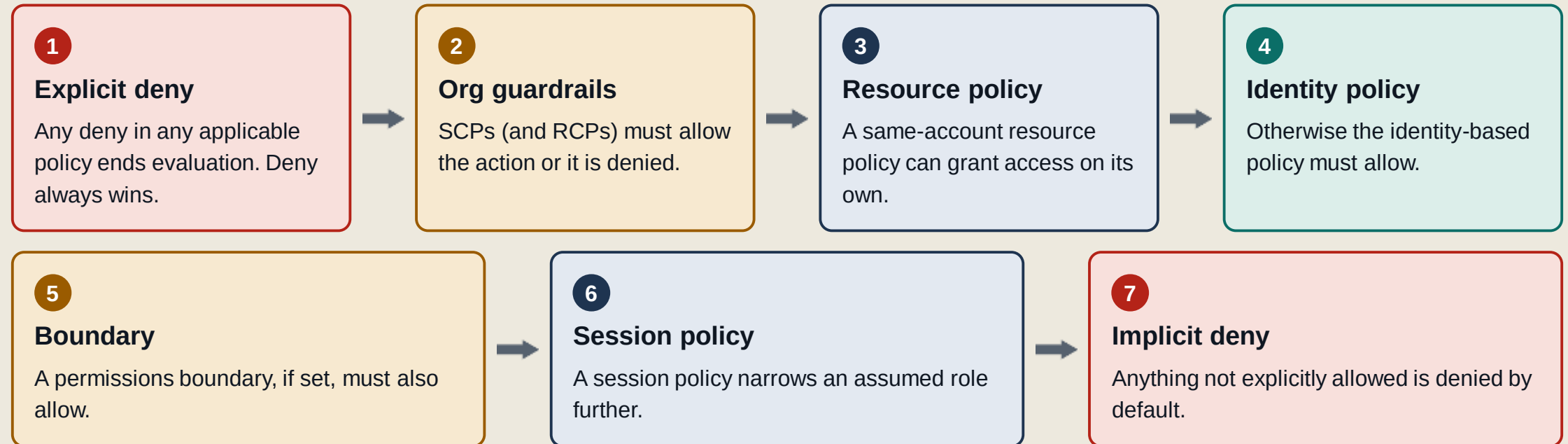
MODELS

Three clouds, three IAM designs

	AWS	Azure	GCP
Identity source	IAM users, roles, Identity Center	Microsoft Entra ID tenant	Cloud Identity, Workspace, federation
Grant unit	JSON policies attached to identities or resources	Role assignments (role, principal, scope)	Role bindings on a resource
Hierarchy	Organization, OU, account	Tenant, management group, subscription, RG	Organization, folder, project
Guardrail	SCP, RCP, permissions boundary	Azure Policy, deny assignments	Org policy, IAM deny policies
Workload identity	IAM roles, instance profiles, IRSA	Managed identity, workload identity federation	Service accounts, WIF
Temporary elevation	STS, Identity Center sets	PIM eligible roles	IAM Conditions, PAM

Key idea Same goals, different mechanics. AWS is policy-first, Azure is assignment-first and GCP is binding-first. Learn each model natively.

How AWS evaluates a request



Key idea Effective permissions are the intersection of ceilings (SCP, boundary, session) and grants, minus explicit denies. Cross-account needs both sides to allow.

A tightly scoped identity policy

identity-policy.json

```
{ "Version": "2012-10-17",  
  "Statement": [{  
    "Sid": "FinanceReportsReadOnly",  
    "Effect": "Allow",  
    "Action": ["s3:GetObject"],  
    "Resource": "arn:aws:s3:::acme-reports/finance/*",  
    "Condition": {  
      "StringEquals": { "aws:PrincipalTag/team": "finance" },  
      "Bool": { "aws:MultiFactorAuthPresent": "true" }  
    }  
  }  
}]  
}
```

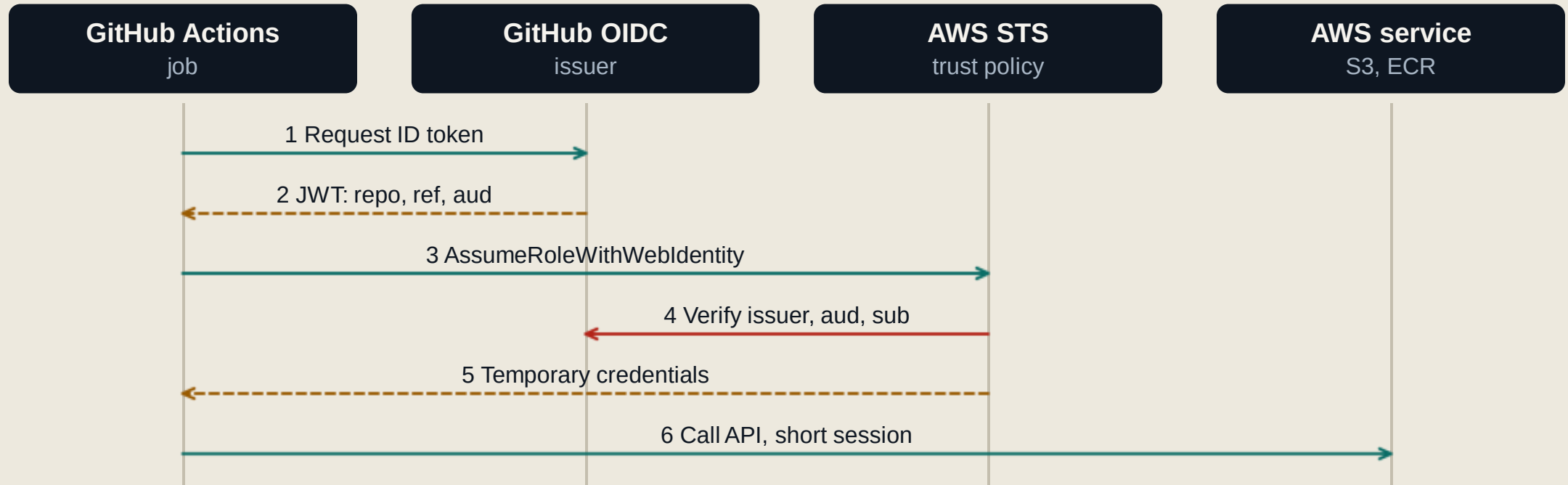
What makes it narrow

- One action, one prefix, no wildcards
- Tag check (ABAC) scales across teams

Add for services

- aws:SourceVpce for network binding
- Separate write and delete policies

CI/CD to AWS without stored keys



Key idea The trust policy must pin sub (repo and branch or environment) and aud. A wildcard sub lets any repository in the provider assume the role.

The OIDC trust policy that matters

role-trust-policy.json

```
{ "Effect": "Allow",  
  "Principal": { "Federated":  
    "arn:aws:iam::111122223333:oidc-provider/token.actions.githubusercontent.com" },  
  "Action": "sts:AssumeRoleWithWebIdentity",  
  "Condition": {  
    "StringEquals": {  
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",  
      "token.actions.githubusercontent.com:sub": "repo:acme/payments:ref:refs/heads/main"  
    }  
  }  
}
```

Anti-pattern Using StringLike with "repo:acme/*" or omitting sub trusts every repository in the organisation, including forks and future ones. Pin repository and environment.

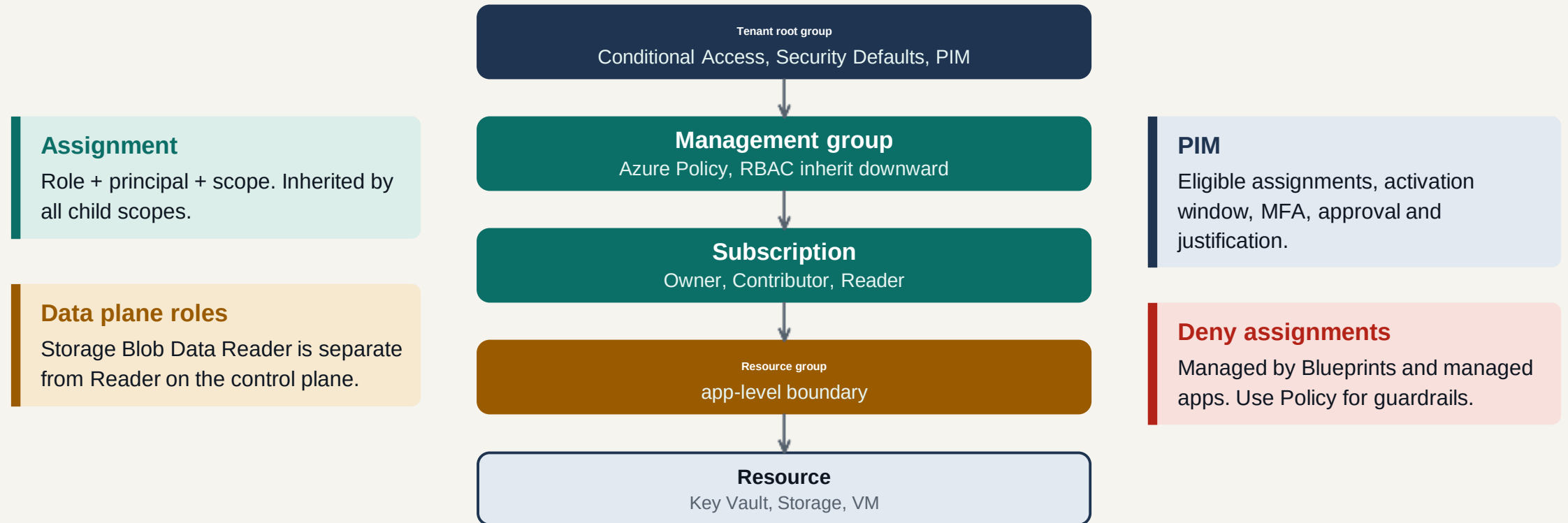
Service control policies as mandatory guardrails

scp-guardrails.json

```
{ "Version": "2012-10-17", "Statement": [
  { "Sid": "ProtectLogging", "Effect": "Deny",
    "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
      "config:StopConfigurationRecorder"], "Resource": "*" },
  { "Sid": "NoLeavingOrg", "Effect": "Deny",
    "Action": "organizations:LeaveOrganization", "Resource": "*" },
  { "Sid": "RegionLock", "Effect": "Deny",
    "NotAction": ["iam:*", "sts:*", "organizations:*", "support:*"],
    "Resource": "*", "Condition": { "StringNotEquals": {
      "aws:RequestedRegion": ["eu-west-2", "eu-west-1"] } } }
]
```

Watch out SCPs never grant permissions, they only cap them. They do not affect the management account. Exempt global services in region locks and test in a sandbox OU first.

Azure RBAC and inherited scope



Azure: scoped assignment with an ABAC condition

Scope low

Prefer resource or resource-group scope over subscription.

Conditions

Role assignment conditions (ABAC) limit data access by container, tag or path.

Custom roles

Compose from Actions and DataActions, list AssignableScopes, no wildcards.

azure cli

```
COND='@Resource[Microsoft.Storage/storageAccounts/'  
COND+='blobServices/containers:name] StringEquals "reports"'
```

```
az role assignment create \  
  --assignee-object-id $OBJ_ID \  
  --assignee-principal-type ServicePrincipal \  
  --role "Storage Blob Data Reader" \  
  --scope $STORAGE_ACCOUNT_ID \  
  --condition-version 2.0 \  
  --condition "$COND"  
  
# who holds privileged roles at subscription scope?  
az role assignment list --scope /subscriptions/$SUB \  
  --role Owner --include-inherited -o table
```

GCP: hierarchy, bindings and impersonation

Hierarchy

Organization, folder, project. Policies inherit downward and are additive. IAM deny policies and org policies add ceilings.

Service accounts

Prefer attached SAs over exported keys. Disable key creation with an org policy. Use short-lived tokens via impersonation.

Conditions

Bind roles with request.time, resource.name or tags. Expiring bindings implement JIT for on-call.

Workload Identity Federation

Exchange external OIDC or SAML tokens for short-lived GCP credentials. No JSON keys in CI.

Watch out `iam.serviceAccounts.actAs` and `getAccessToken` on a powerful SA equal that SA's power. Treat them as privileged.

GCP: expiring binding for on-call access

Time-bound

IAM Conditions make the grant expire without cleanup.

Audit

Cloud Audit Logs record Admin Activity by default.
Enable Data Access logs for sensitive services.

Analyse

Policy Analyzer and Recommender show unused roles and effective access.

gcloud

```
COND='expression=request.time < '  
COND+='timestamp("2026-10-01T00:00:00Z"),title=oncall-oct'
```

```
gcloud projects add-iam-policy-binding prod-payments \  
  --member="group:sre-oncall@example.com" \  
  --role="roles/compute.instanceAdmin.v1" \  
  --condition="$COND"
```

```
# block key creation across the organisation  
gcloud resource-manager org-policies enable-enforce \  
  iam.disableServiceAccountKeyCreation \  
  --organization=$ORG_ID
```

Kubernetes RBAC: namespaced, verb-specific

Prefer

Role and RoleBinding in a namespace over ClusterRole and ClusterRoleBinding.

Avoid

Wildcards, cluster-admin, and the escalate, bind and impersonate verbs.

Sensitive

get or list on secrets, create on pods, and pods/exec are effectively broad access.

yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: deploy-reader, namespace: payments }
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch"]
---
kind: RoleBinding
metadata: { name: sre-read, namespace: payments }
subjects:
- { kind: Group, name: sre-oncall, apiGroup: rbac.authorization.k8s.io }
roleRef: { kind: Role, name: deploy-reader,
          apiGroup: rbac.authorization.k8s.io }
```

ESCALATION

Common privilege escalation paths in cloud IAM

Cloud	Permission	Abuse	Mitigation
AWS	iam:CreatePolicyVersion, AttachUserPolicy	Write yourself admin	Deny in SCP, restrict to break-glass
AWS	iam:PassRole with ec2 or lambda create	Run code as a stronger role	Limit PassRole resource and passed-to service
AWS	sts:AssumeRole to broad trust	Chain roles up	Pin principals, require conditions
Azure	User Access Administrator, Owner	Assign yourself any role	PIM, alert on assignments
Azure	App admin adds a secret to a service principal	Become that app	Restrict app admin roles, review credentials
GCP	iam.serviceAccounts.actAs, setIamPolicy	Impersonate or grant admin	Custom roles, deny policies, alerts

Key idea Any permission that can write IAM, pass a role or impersonate is an admin permission in disguise. Treat it that way.

Confused deputy and trust boundaries

Third-party access

Require an ExternalId when a vendor assumes your role, so another customer cannot trick the vendor into using your role.

Service principals

Add aws:SourceArn and aws:SourceAccount when a service (SNS, S3, CloudTrail) acts on your behalf.

Organisation trust

Use aws:PrincipalOrgID in resource policies rather than account root principals with no conditions.

Key idea Every trust relationship is a grant. Enumerate them, and review who can assume what from where.

A cloud entitlement review, in order

1	Inventory every identity, human, workload and third party, across accounts	DISCOVER
2	Compute effective permissions including groups, boundaries and SCPs	RESOLVE
3	Flag admin-equivalent and escalation-capable permissions	RISK
4	Compare with observed use and mark unused permissions	USAGE
5	Generate tighter policies and route to the owners in pull requests	FIX
6	Prevent regressions with policy-as-code checks in CI	GUARD

Key idea This is the CIEM loop. Tools help, but ownership and pipelines make it stick.

CHECKPOINT 7

Check your understanding

Question	Model answer
Which wins in AWS evaluation, allow or deny?	An explicit deny in any applicable policy always wins.
Does an SCP grant permissions?	No. It only caps what identities in the account can be granted.
What must an OIDC trust policy pin?	The audience and a specific sub (repository and branch or environment).
Why is actAs powerful in GCP?	It lets a principal use a service account's permissions.
Why prefer Role over ClusterRole in Kubernetes?	It limits blast radius to a namespace.

PART 8

Directories, Active Directory and hybrid identity

The identity backbone of most enterprises, its classic attack paths, and how to tier, harden and connect it to the cloud.

IN THIS PART

- Active Directory building blocks
- Tiering and the Enterprise Access Model
- Attack paths and hardening
- Delegation and service accounts
- Hybrid sync, PHS, PTA and federation
- Conditional Access and IdP resilience

Active Directory in one slide

Forest and domain

Forest is the security boundary. Domains partition administration and replication. Trusts extend authentication between them.

Domain controllers

Hold NTDS.dit, run the Kerberos KDC and LDAP. Full replicas, so compromise of one is compromise of all.

Objects and ACLs

Every object has a security descriptor. Rights like GenericAll and WriteDacl are attack primitives.

Group Policy

Central config for machines. Whoever edits a GPO linked to an OU controls every machine in it.

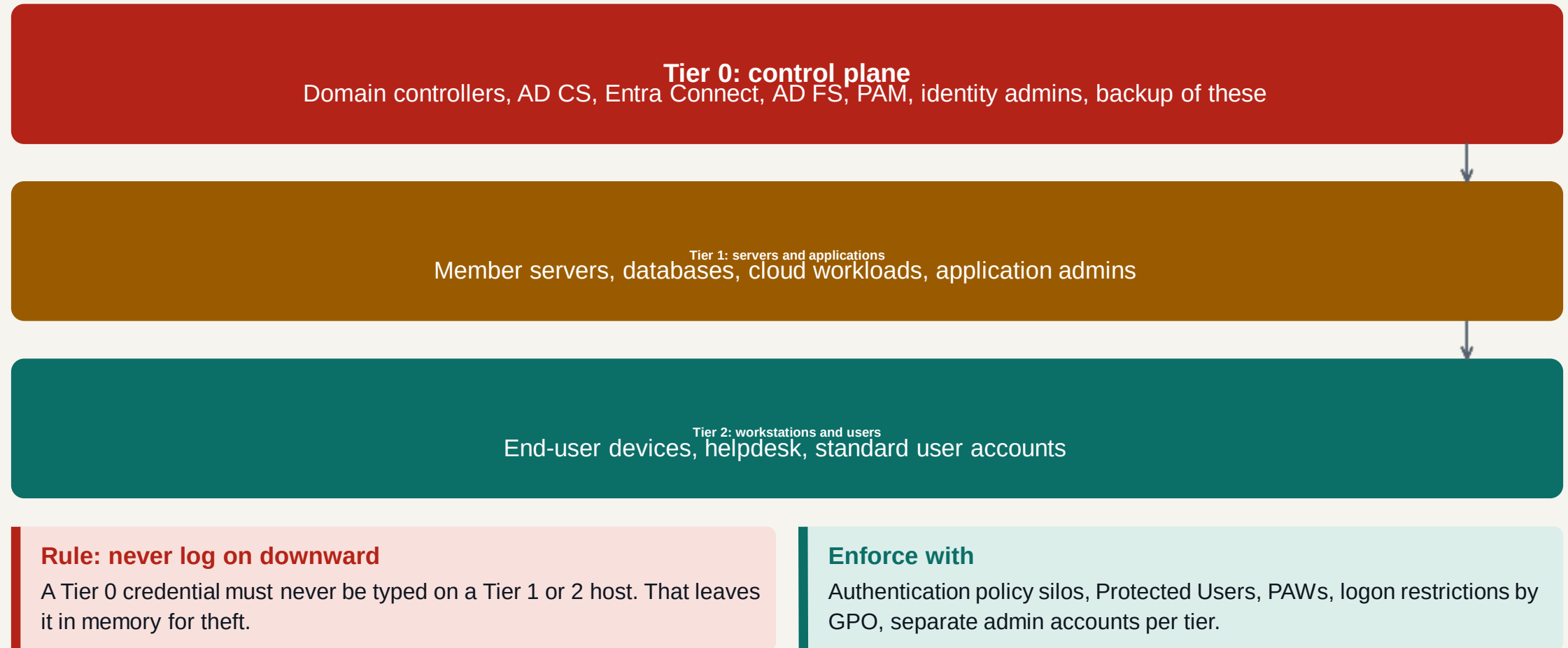
AD CS

The estate CA. Weak templates let low-privilege users request certificates as anyone.

Key idea AD is the crown jewel. Its security depends on ACLs, delegation and configuration more than on patching.

TIERING

Administrative tiering: contain credential theft



Classic Active Directory attack paths

Technique	How it works	Defence
Kerberoasting	Any user requests service tickets for SPNs and cracks them offline	gMSA, long service passwords, AES only, alert on RC4 requests
AS-REP roasting	Accounts without pre-auth expose crackable data	Require pre-authentication, audit flags
DCSync	Account with replication rights pulls password hashes	Restrict replication rights, alert on 4662 from non-DCs
Delegation abuse	Unconstrained delegation caches TGTs of visitors	Remove unconstrained, use constrained or RBCD, sensitive flag
ACL abuse	GenericAll or WriteDacl on privileged objects	BloodHound review, reduce delegated rights
AD CS ESC1 and friends	Template lets requester set the subject alternative name	Audit templates, remove enrollee-supplied SAN, patch CA

Reality Attackers chain these. Most paths to Domain Admin use misconfiguration, not zero-days. Run BloodHound and PingCastle defensively.

AD hardening commands you can run today

Find

Accounts with SPNs are Kerberoast targets. Review age and privilege.

Protect

Protected Users blocks NTLM, RC4 and delegation for members.

Replace

gMSAs rotate long random passwords automatically.

powershell

```
# Kerberoastable user accounts
```

```
Get-ADUser -Filter {ServicePrincipalName -ne "$null"} `
  -Properties ServicePrincipalName,PasswordLastSet,MemberOf |
  Select Name,PasswordLastSet,ServicePrincipalName
```

```
# harden privileged accounts
```

```
Add-ADGroupMember -Identity "Protected Users" -Members alice-adm
```

```
# group managed service account
```

```
New-ADServiceAccount -Name gmsa-sql `
  -DNSHostName gmsa-sql.corp.example `
  -PrincipalsAllowedToRetrieveManagedPassword "SQL-Servers"
```

Kerberos delegation, ranked by danger

Unconstrained

The service stores the caller's TGT. Anyone who reaches it can be impersonated everywhere. Never on servers that admins visit. Do not use it.

Constrained

The service may impersonate users only to listed SPNs. Better, but the service still holds strong power. Protocol transition widens it.

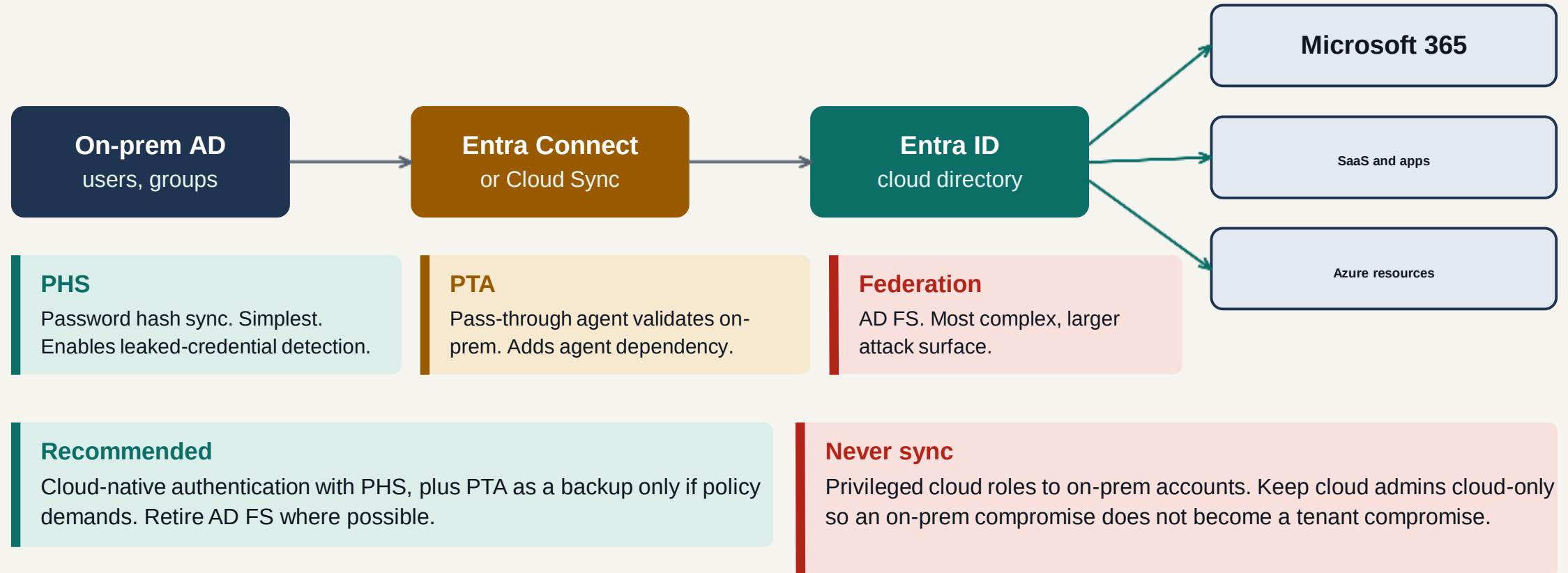
Resource-based (RBCD)

The target decides who may delegate to it. Easier to scope, but write access to msDS-AllowedToActOnBehalfOfOtherIdentity is a takeover primitive.

Key idea Mark privileged users "Account is sensitive and cannot be delegated" and place them in Protected Users.

HYBRID

Hybrid identity topology and sign-in options



Attack paths that cross the boundary

Sync account theft

The Entra Connect account can write passwords and objects. Treat the server as Tier 0, monitor its account, restrict logon.

Golden SAML

A stolen AD FS token-signing key mints valid assertions for any user. Protect the key, monitor certificate export, migrate off AD FS.

Cloud to on-prem

Cloud admin roles reaching on-prem via agents, and hybrid-joined devices with privileged sessions. Separate admin identities.

Trusts and multi-tenant

Guest access, B2B trust and cross-tenant sync widen exposure. Restrict inbound and outbound settings.

Key idea The trust boundary between on-prem and cloud is a design decision. Assume compromise on one side and make sure it does not become both.

A Conditional Access baseline that enforces least privilege

Policy	Applies to	Grant control
Block legacy authentication	All users	Block
MFA for all users	All users, all apps	Require MFA, prefer authentication strength
Phishing-resistant MFA for admins	Privileged directory roles	Passkey, FIDO2 or certificate authentication strength
Compliant device for sensitive apps	Finance, HR, admin portals	Require compliant or hybrid-joined device
Risk-based sign-in	Medium or high sign-in risk	Require MFA or block
Admin session limits	Admins	Sign-in frequency and no persistent browser session

Watch out Test in report-only mode. Exclude break-glass accounts. Track exclusions, they become permanent holes.

Use narrow directory roles, not Global Administrator

Split by task

User Administrator, Helpdesk Administrator, Authentication Administrator, Exchange Administrator, Application Administrator. Choose the smallest that fits.

Limit the top

Keep Global Administrators few, cloud-only and eligible through PIM. Microsoft recommends fewer than five.

Scope

Administrative units and restricted management units limit roles to a subset of users or groups.

Key idea Roles that can reset credentials or add app secrets are as dangerous as global admin for the accounts they control.

Designing for identity provider outages

Know the dependencies

Map which systems fail when the IdP is down: VPN, PAM, ticketing, cloud consoles, monitoring. Avoid circular dependencies.

Redundancy

Multi-region IdP, redundant sync, fall-back authentication for critical operations, tested break-glass.

Cache and degrade

Cached credentials and offline tokens for critical systems, with tight expiry and clear limits.

Key idea Availability is a security property. If the IdP is unreachable, teams will bypass controls. Design the safe fallback in advance.

CHECKPOINT 8

Check your understanding

Question	Model answer
Why is a domain controller compromise total?	It holds all password material and the Kerberos keys, so an attacker can mint any identity.
What does tiering prevent?	Privileged credentials being exposed on lower-trust hosts where they can be stolen.
Why prefer gMSA?	Automatic long random password rotation removes Kerberoasting and reuse risks.
Why keep cloud admins cloud-only?	So an on-prem AD compromise does not carry into the cloud tenant.
Why use report-only mode for Conditional Access?	It shows the impact before enforcement and avoids lockouts.

PART 9

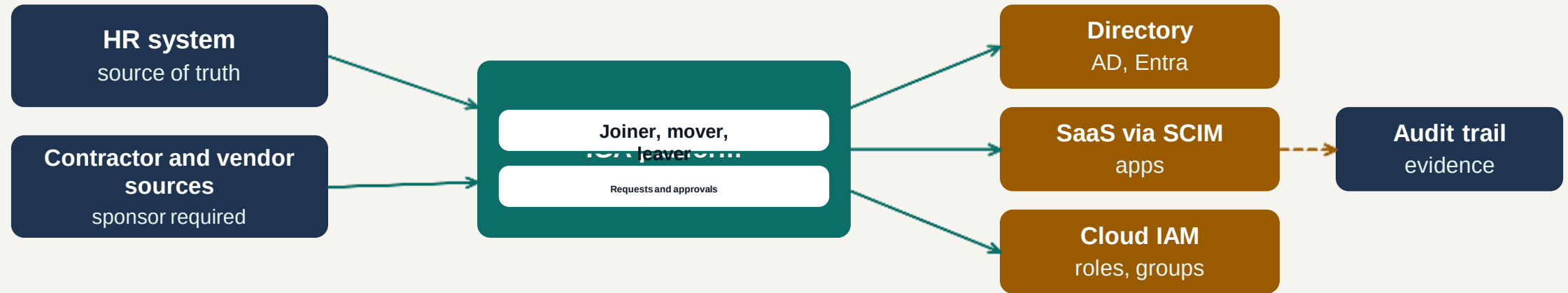
Identity governance and administration

Keeping access correct over time: lifecycle, requests, certification, role engineering and audit evidence.

IN THIS PART

- IGA architecture and lifecycle
- Requests, approvals and birthright access
- Access certification that works
- Role mining with data
- Compliance mapping and evidence
- Governing non-human and third-party identities

IGA architecture: sources, policy, targets



Authoritative sources

HR drives workforce identity. Sponsors drive contractors. Never let managers create accounts by hand.

Policy in the middle

Birthing rules, SoD checks and approval workflows live in the IGA, not in each app.

Closed loop

Reconcile actual entitlements in targets against approved state. Flag drift and orphans.

Key idea IGA turns access into a governed lifecycle: authoritative source in, policy applied, provisioned everywhere, evidenced always.

Joiner, mover, leaver: rules and service levels

Event	Trigger	Required action	Target
Joiner	HR hire record, sponsor for contractors	Create identity, birthright role, mailbox, MFA enrolment	Ready on day one
Mover	Job, department or location change in HR	Remove old-role access, grant new-role access, re-approve exceptions	Within 24 hours
Leaver	Termination date in HR	Disable sign-in, revoke sessions and tokens, remove access, transfer data and ownership	Immediately, at end of contract time
Long absence	Leave status	Suspend or reduce, restore on return	By policy
Contractor end	Sponsor or contract end date	Auto-expire, require renewal by sponsor	Fixed expiry at creation

Watch out Movers are the biggest source of privilege creep. People accumulate access from every previous role. Design mover as remove-then-add.

REQUESTS

Birthright, requestable and privileged access

Birthright

Access everyone in a role gets automatically, by department, level and location. Keep it small and identical across similar staff.

Requestable

A catalogue with clear descriptions, risk ratings, owner and default duration. Manager plus resource-owner approval for sensitive items.

Privileged

Never a standing request. Eligible through JIT, with security approval, session recording and a short lifetime.

Key idea Make the safe path fast. If requests take a week, people ask for broad access and hold it forever.

Designing an access review that gets read



Reality A review that approves 99 percent of items in seconds is not a control. Reduce volume, add context, use usage data.

Role mining with hierarchical clustering

Idea

Represent each user as a set of entitlements and cluster similar sets. Frequent common entitlements become candidate roles.

Then

Validate with owners. Roles must reflect job function, not merely observed overlap.

Caution

Mining freezes today's over-privilege. Remove unused entitlements first.

python

```
import pandas as pd
from scipy.spatial.distance import pdist
from scipy.cluster.hierarchy import linkage, fcluster

df = pd.read_csv("entitlements.csv") # user, entitlement
M = (df.assign(v=1)
     .pivot_table(index="user", columns="entitlement",
                  values="v", fill_value=0).astype(bool))
Z = linkage(pdist(M.values, "jaccard"), "average")
cl = fcluster(Z, t=0.4, criterion="distance")

for c in sorted(set(cl)):
    grp = M[cl == c]
    core = grp.columns[grp.mean() >= 0.8] # held by 80%+
    print(c, len(grp), list(core))
```

Mapping identity controls to frameworks

Framework	Relevant controls	What auditors look for
NIST SP 800-53	AC-2, AC-5, AC-6, IA-2, IA-5	Account management, SoD, least privilege, MFA
ISO/IEC 27001:2022	A.5.15 to A.5.18, A.8.2, A.8.3	Access control policy, provisioning, privileged access
PCI DSS v4.0	Requirements 7 and 8	Need-to-know access, MFA, six-month account reviews
SOC 2	CC6.1 to CC6.3	Logical access, provisioning, removal, periodic review
SOX ITGC	Access to programs and data	Approvals, terminated users, privileged access reviews
GDPR	Art. 5(1)(f), 25, 32	Integrity and confidentiality, access by need, security by design

Key idea Build one control set and map it to many frameworks. Do not build per-framework processes.

EVIDENCE

Audit evidence you should produce automatically

1	Joiner, mover, leaver samples with timestamps from HR event to removal	LIFECYCLE
2	Complete access review results with reviewer, decision and provisioning proof	REVIEWS
3	Privileged access list, approvals, session recordings and elevation logs	PRIVILEGED
4	MFA and Conditional Access configuration snapshots and exception lists	AUTHN
5	SoD rule set, violations found, mitigations and approved exceptions	SOD

Key idea If evidence needs a manual spreadsheet at audit time, the control is fragile. Automate collection continuously.

Who owns what in identity governance

Activity	Business owner	IAM team	Security	Audit
Define access policy and roles	C	R	A	I
Approve access requests	R	I	C	I
Run access reviews	R	A	C	I
Operate the IdP and PAM	I	R	A	I
Detect and respond to identity threats	I	C	R	I
Test controls and report	I	C	C	R

Note R responsible, A accountable, C consulted, I informed. Adjust to your operating model. The point is that every row has exactly one A.

Governing workload, third-party and AI identities

Ownership

Every service account, key and app registration has a named owner, purpose and expiry. Orphaned identities are disabled after notice.

Inventory and hygiene

Continuous discovery of keys, tokens and app credentials. Report age, last use and scope. Rotate or retire.

Third parties

Sponsor, contract end date, MFA, narrow scope, session recording for privileged vendors. Review quarterly.

Reality Non-human identities outnumber humans by large multiples in most cloud estates, yet receive far less governance. Close that gap first.

CHECKPOINT 9

Check your understanding

Question	Model answer
Why are movers a special risk?	Access accumulates across roles. Without remove-then-add, privilege creep grows.
What makes an access review meaningful?	Risk-based scope, right reviewer, usage context and easy revoke with verified removal.
Why not mine roles from raw current access?	It encodes existing over-privilege. Right-size first, then mine.
Why map controls to several frameworks?	One control set with many mappings avoids duplicate work and drift.
Who owns a service account?	A named human owner with a purpose and expiry recorded.

PART 10

Detecting identity attacks

Telemetry, ATT&CK mapping, detection queries and response playbooks for the way modern intrusions actually move: through identities.

IN THIS PART

- Identity telemetry sources
- ATT&CK techniques and Windows events
- Detection queries in KQL and Sigma
- Token theft and MFA fatigue response
- Deception and ITDR metrics
- A compromised-admin playbook

Identity telemetry to collect and retain

Source	Key records	Why it matters
Entra ID / IdP	Sign-in logs, audit logs, risk detections, consent grants	Password spray, token replay, new MFA methods, app consent abuse
Active Directory	Security events 4624, 4625, 4662, 4672, 4768, 4769, 4728, 4732	Kerberoasting, DCSync, privileged group changes, lateral movement
Cloud control plane	CloudTrail, Azure Activity Log, GCP Audit Logs	IAM changes, role assumption, key creation, log tampering
PAM and vault	Checkouts, session recordings, break-glass use	Privileged misuse, anomalous elevation
Kubernetes	Audit logs: exec, secrets get, rolebinding changes	Cluster privilege escalation and secret theft
SaaS and email	Admin audit, OAuth grants, mailbox rules, API tokens	Business email compromise, data exfiltration

Key idea Log identity events centrally, retain at least 12 months where you can, and protect the log pipeline from the identities it monitors.

MITRE ATT&CK techniques for identity attacks

ID	Technique	Signal to detect
T1078	Valid Accounts (incl. .004 Cloud)	Impossible travel, new device, unusual resource access
T1110.003	Password Spraying	Many users, few passwords, one source, low volume per user
T1621	MFA Request Generation	Bursts of denied or unanswered MFA prompts
T1558.003	Kerberoasting	Bursts of 4769 with RC4 (0x17) for many SPNs
T1003.006	DCSync	Replication requests from non-DC hosts
T1550	Use Alternate Authentication Material	Token reuse from a new IP or user agent
T1606.002	Forge Web Credentials: SAML Tokens	Assertions with no matching IdP sign-in
T1098	Account Manipulation	New credentials on apps, roles added, MFA method added

Key idea Map detections to ATT&CK to track coverage. Cover credential access, persistence and privilege escalation first.

Windows Security events every identity team should know

Event	Meaning	Look for
4624 / 4625	Successful and failed logon (with logon type)	Type 10 or 3 from unusual sources, bursts of failures
4672	Special privileges assigned to new logon	Admin logons on workstations or unusual hosts
4768 / 4769	Kerberos TGT and service ticket requests	RC4 tickets, unusual SPN volume, ticket without a preceding TGT
4662	Operation on a directory object	Replicating Directory Changes GUIDs from non-DCs
4720 / 4728 / 4732 / 4756	Account created, added to a group	Additions to Domain Admins, Backup Operators, Tier 0 groups
4740	Account lockout	Lockouts across many accounts in a short time (spray or stuffing)

KQL: password spray in Entra sign-in logs

Signal

Many distinct users failing from one IP with a small number of attempts each.

Tuning

Exclude known corporate egress and scanners.
Baseline by tenant size.

Response

Block the IP or named location, force reset for successful matches.

kql

```
SigninLogs
| where TimeGenerated > ago(1h)
| where ResultType in ("50126", "50053")
| summarize Users = dcount(UserPrincipalName),
              Attempts = count(),
              Apps = make_set(AppDisplayName, 5)
              by IPAddress, bin(TimeGenerated, 10m)
| where Users > 10 and Attempts / Users < 4
| order by Users desc
```

KQL: Kerberoasting from domain controller events

Signal

A single account requests tickets for many service names using RC4.

Why RC4

Attack tools request downgraded ciphers, and AES-only estates make RC4 rare.

Tune

Exclude legacy applications you know need RC4, then remove RC4 domain-wide.

kql

```
SecurityEvent
| where EventID == 4769
| where TicketEncryptionType == "0x17"
| where ServiceName !endswith "$"
| where ServiceName != "krbtgt"
| summarize Services = dcount(ServiceName),
    List = make_set(ServiceName, 10)
    by TargetUserName, IpAddress,
    bin(TimeGenerated, 10m)
| where Services > 5
```

Sigma: sensitive IAM changes in CloudTrail

Portable

Sigma rules compile to many SIEMs, so detections live in Git like code.

Scope

Alert on creation of access keys, policy attachments and trust policy edits, outside the pipeline identity.

Enrich

Add owner, environment and change ticket for triage.

`sigma`

`title: IAM privilege change outside automation`

`status: experimental`

`logsource: { product: aws, service: cloudtrail }`

`detection:`

`sel:`

`eventSource: iam.amazonaws.com`

`eventName:`

- `- CreateAccessKey`
- `- AttachUserPolicy`
- `- PutUserPolicy`
- `- UpdateAssumeRolePolicy`

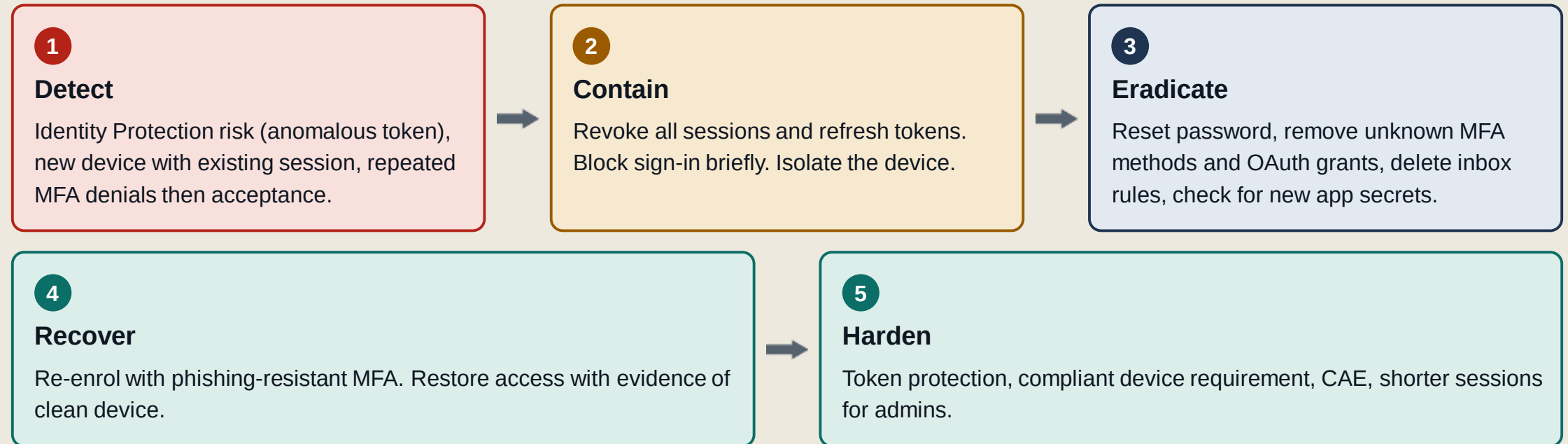
`filter_ci:`

`userIdentity.arn|contains: "role/terraform-apply"`

`condition: sel and not filter_ci`

`level: high`

Responding to token theft and MFA fatigue



Key idea Password reset alone does not end a stolen-session incident. Always revoke tokens and review persistence.

RESPONSE

Response commands: revoke sessions and cut access

Entra

Revoke refresh tokens, list and remove MFA methods, review consented apps.

AWS

Deactivate keys, then invalidate role sessions with a time-conditioned deny.

Practice

Script these in a runbook. Test them before you need them.

response

Entra ID via Microsoft Graph PowerShell

```
Revoke-MgUserSignInSession -UserId alice@corp.example
```

```
Get-MgUserAuthenticationMethod -UserId alice@corp.example
```

AWS: disable a leaked key

```
aws iam update-access-key --user-name alice \  
--access-key-id AKIA... --status Inactive
```

AWS: revoke sessions issued before now (attach to role)

```
aws iam put-role-policy --role-name app-role \  
--policy-name RevokeOldSessions \  
--policy-document file://deny-before.json
```

Deception and ITDR: catch what logs miss

Honeytokens

Fake access keys, service accounts with attractive SPNs, canary documents. Any use is high confidence. Alert at once.

Identity threat detection and response

ITDR tools baseline identity behaviour, watch AD and IdP together, and detect DCSync, golden ticket and token abuse.

Detection as code

Rules in Git with tests and sample logs. Track coverage against ATT&CK and false positive rate.

Metric Mean time to detect and to contain identity incidents, detection coverage of your top attack paths, and percentage of alerts with a runbook.

Compromised privileged account: first response

1	Confirm alert, identify the account, sessions, source IPs and resources touched	TRIAGE
2	Disable sign-in, revoke sessions and tokens, rotate secrets the account could read	CONTAIN
3	Review audit logs for new credentials, roles, OAuth apps, forwarding rules and policy changes	SCOPE
4	Remove persistence, patch the root cause, reset dependent service accounts	ERADICATE
5	Restore access with hardened auth, monitor closely for 30 days, run a blameless review	RECOVER

Watch out Assume the attacker created a second way back in. A privileged compromise is not closed until persistence is ruled out.

CHECKPOINT 10

Check your understanding

Question	Model answer
Which Windows event shows a Kerberos service ticket request?	4769. RC4 encryption type 0x17 for many SPNs suggests Kerberoasting.
What distinguishes password spraying from brute force?	Few passwords across many accounts, keeping per-account failures low.
Why is a password reset not enough after token theft?	Existing sessions and refresh tokens stay valid until revoked.
Why use honeytokens?	Legitimate use is nil, so any access is high-confidence detection.
What does Sigma provide?	Vendor-neutral detection rules that live in version control.

PART 11

Building the programme

From assessment to target architecture: sequencing, migration, lessons from public incidents and how to report progress.

IN THIS PART

- Maturity model and assessment
- Target architecture
- A 12-month roadmap
- Quick wins and migration playbooks
- Lessons from real incidents
- Metrics and failure modes

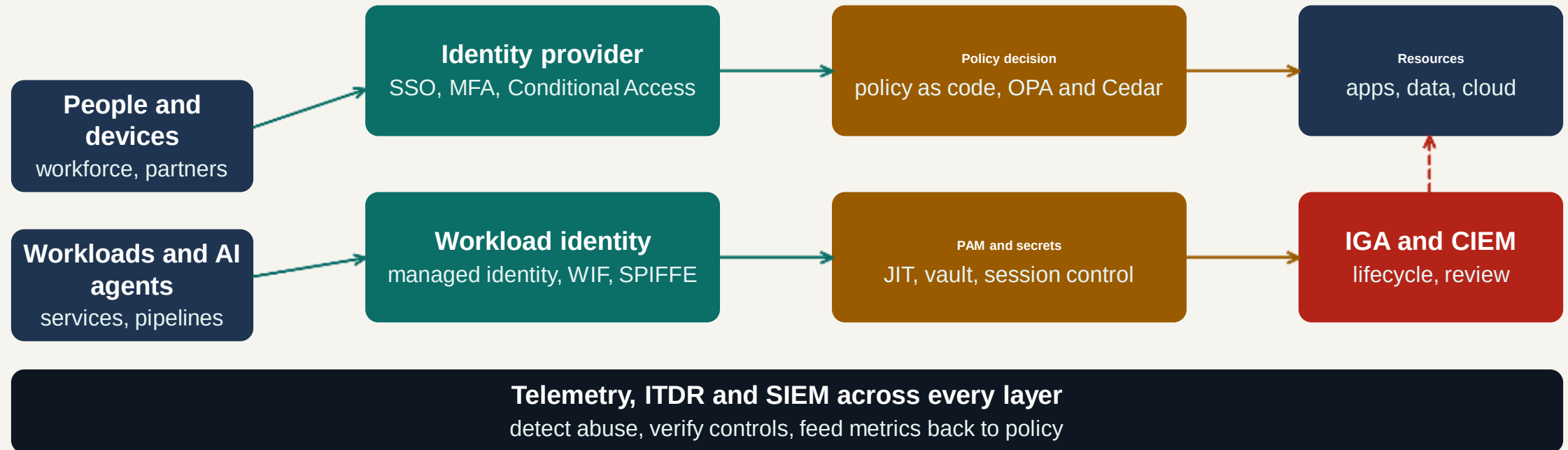
MATURITY

An IAM and least privilege maturity model

Level	Name	Characteristics
1	Ad hoc	Local accounts, shared admin, manual joiners and leavers, no inventory
2	Managed	Central directory, SSO and MFA for most users, basic reviews, admin accounts separated
3	Defined	IGA lifecycle, PAM with vaulting, cloud guardrails, documented roles, logging in SIEM
4	Measured	Usage-based right-sizing, JIT, phishing-resistant MFA, detection coverage tracked
5	Adaptive	Zero standing privilege, policy as code, continuous verification, automated response

Key idea Assess by domain, not one score. Authentication may be at level 4 while governance is at level 2. Fund the weakest link that attackers can reach.

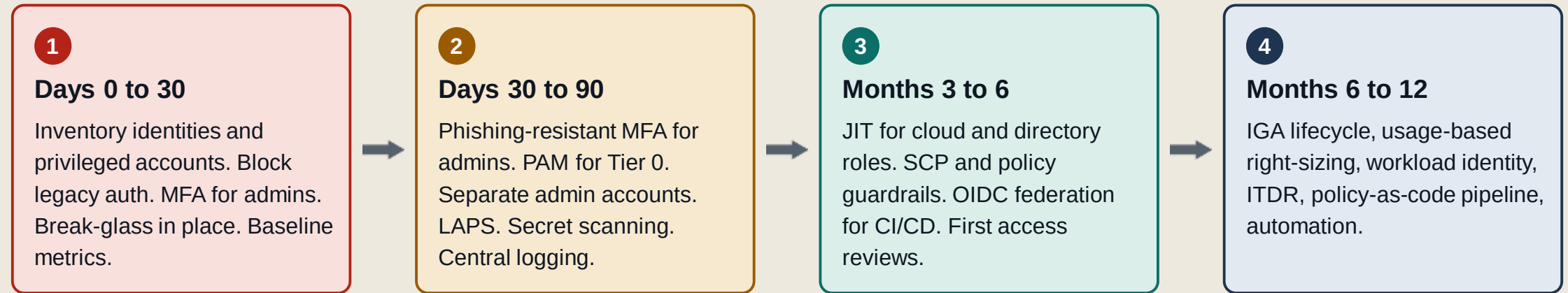
Target architecture: identity as the control plane



Key idea Every request flows through identity, policy and logging. Governance keeps it correct. Detection proves it.

ROADMAP

A 12-month roadmap



Key idea Sequence by attacker path: protect Tier 0 and admin identities first, then broad workforce, then workloads, then optimise.

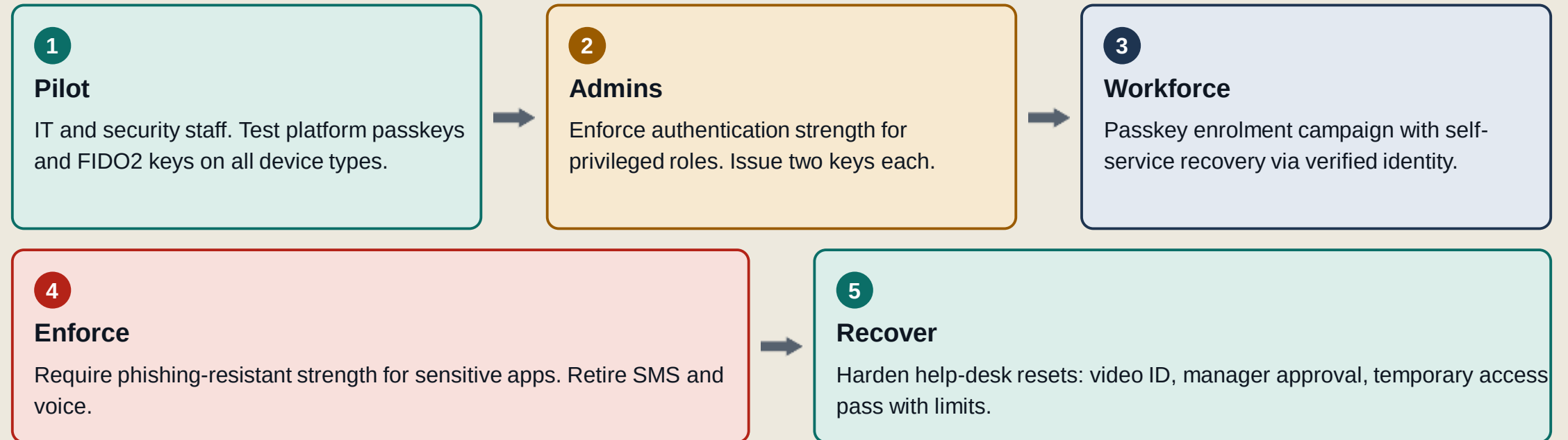
QUICK WINS

Highest-value first moves

Action	Effort	Risk reduced
Block legacy authentication protocols	Low	Password spray and credential stuffing bypassing MFA
Phishing-resistant MFA for privileged roles	Medium	Admin takeover through phishing and AiTM
Disable or vault long-lived cloud access keys	Medium	Leaked key compromise
Deploy LAPS and unique local admin passwords	Low	Lateral movement by pass-the-hash
Protect logging and backups with SCP and immutability	Low	Anti-forensics and ransomware recovery
Remove standing Global Admin and Owner assignments	Medium	Tenant-wide takeover

Watch out Quick wins are quick because the risk is obvious. Do them before building tooling that takes a year.

Rolling out phishing-resistant MFA



Reality Help-desk social engineering bypasses strong MFA. Recovery is part of the authentication design.

CASE STUDIES

Lessons from public identity incidents (summaries of public reporting)

Incident	What failed	Lesson
Capital One, 2019	A web app flaw let an attacker obtain credentials of an over-permissive cloud role	Scope roles narrowly, harden metadata access, monitor unusual role use
SolarWinds, 2020	Attackers forged SAML assertions after stealing a federation signing key	Protect token-signing keys, monitor federation, prefer cloud auth
Uber, 2022	MFA fatigue against a contractor, then admin credentials in a script	Phishing-resistant MFA, no hard-coded privileged secrets
Microsoft Storm-0558, 2023	A stolen signing key and a validation flaw let forged tokens reach mailboxes	Key custody and rotation, strict issuer and audience validation
MGM, 2023	Help-desk social engineering reset credentials and MFA	Strong recovery and verification for privileged staff

Note Details are drawn from public reports and vendor statements and may be refined over time. Use them to drive discussion, not as legal findings.

What to report upward

0

standing Global Admin or Owner assignments is the target

100%

of admins on phishing-resistant MFA

< 4 h

from leaver event to all access removed

90 d

maximum age of any active credential or key

Metric	Why executives care
Share of identities with unused permissions	Direct measure of attack surface
Mean time to detect and contain identity incidents	Speed limits damage
Access review completion and revocation rate	Audit posture and control effectiveness

FAILURE MODES

Why IAM programmes stall

Tool first

Buying an IGA or PAM suite without process, ownership or data quality.
Fix data and roles first.

Big bang

Attempting to right-size everything at once and causing outages. Move owner by owner.

No owners

Access reviews sent to people who do not know what the access is.
Assign resource owners.

Security only

Business units see IAM as friction. Co-design, make requests fast, share metrics.

Key idea IAM is a socio-technical system. Ownership, incentives and usability decide success more than product choice.

CHECKPOINT 11

Check your understanding

Question	Model answer
Why assess maturity by domain?	Weak domains, such as governance, can undermine strong ones. Fund the weakest reachable link.
What should be secured first?	Tier 0 and privileged admin identities, because they control everything else.
Why is recovery part of authentication design?	Attackers use weak help-desk resets to bypass strong MFA.
What is a good metric for standing privilege?	Number of always-active privileged assignments, targeting zero.
Why do tool-first programmes fail?	Without ownership, data quality and process, tooling automates the mess.

PART 12

Hands-on labs and reference

Six labs to build the skills, review checklists for real designs, a glossary and the standards to read next.

IN THIS PART

- Six labs with verification steps
- Authentication and authorization design checklists
- Glossary of core terms
- Standards and further reading

Labs: OIDC login and authorization as code

Lab 1: OIDC with PKCE

Run a local IdP, register a public client, complete the authorization code flow with PKCE, then validate the ID token. **Verify:** tamper with the code_verifier and confirm the token request fails. **Stretch:** add a refresh token with rotation.

Lab 2: Policy as code

Write a Rego policy for tenant-scoped order access, add tests for cross-tenant and expired tokens, and run them in CI. **Verify:** every deny test passes and a mutation of the policy fails a test. **Stretch:** add decision logging.

Tip Use throwaway containers and test data only. Never point labs at production identity systems.

Labs: right-sizing and keyless CI/CD

Lab 3: Right-size a role

In a sandbox account, give a role broad access, run a workload, then use last-accessed data and policy generation to draft a narrower policy. **Verify:** the workload still runs and denied actions are logged. **Stretch:** enforce with a permissions boundary.

Lab 4: OIDC from CI to cloud

Create an OIDC provider and role in a sandbox account, pin repository and branch in the trust policy, then deploy from a workflow with no stored keys. **Verify:** a fork or another branch cannot assume the role.

Tip Attackers test the negative case. So should you: always attempt the request that must fail.

Labs: detection and dynamic secrets

Lab 5: Detect spray and roast

Load sample sign-in and Windows event logs into a SIEM or notebook, implement the spray and Kerberoasting queries, tune out false positives and write a runbook. **Verify:** replay sample attacks and confirm alerts fire.

Lab 6: Dynamic database secrets

Run a database and a secrets manager in containers, configure a dynamic role with a one-hour lease, connect an app, then revoke the lease. **Verify:** the credential stops working immediately after revoke.

Tip Capture screenshots and configs from each lab to build a portfolio of evidence and reusable templates.

Lab environment quick start

Local IdP

Keycloak in dev mode for Lab 1. Not for production.

Secrets manager

Vault in dev mode for Lab 6. Data is in memory only.

Policy engine

OPA CLI for Lab 2. Runs tests with no server.

shell

```
# Lab 1: Keycloak (dev mode)
```

```
docker run -p 8080:8080 \
```

```
-e KC_BOOTSTRAP_ADMIN_USERNAME=admin \
```

```
-e KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
```

```
quay.io/keycloak/keycloak start-dev
```

```
# Lab 6: Vault (dev mode)
```

```
docker run --cap-add=IPC_LOCK -p 8200:8200 \
```

```
-e VAULT_DEV_ROOT_TOKEN_ID=root hashicorp/vault
```

```
# Lab 2: OPA tests
```

```
opa test ./policy -v
```

CHECKLIST

Authentication design review checklist

1	Phishing-resistant MFA for admins and sensitive apps, with hardened recovery	MFA
2	Legacy protocols blocked, NTLM reduced, Kerberos uses AES	PROTOCOLS
3	Sessions and tokens: short lifetimes, sender constraint or device binding where possible	SESSIONS
4	OAuth and OIDC: PKCE, exact redirect matching, audience and issuer validation, minimal scopes	OAUTH
5	Passwords stored with Argon2id or equivalent, breach screening, no forced rotation	PASSWORDS
6	Detection on spray, MFA fatigue, token replay and new MFA methods	DETECT

Use Score each row red, amber or green with an owner and a date. Review quarterly.

CHECKLIST

Authorization and least privilege review checklist

1	Every API checks object and function level access, deny by default, tests for cross-tenant access	AUTHZ
2	No standing admin. JIT with approval and expiry for privileged roles	JIT
3	Roles right-sized from usage data every quarter. No wildcard on sensitive services	SCOPE
4	Guardrails: SCPs, boundaries and org policies protect logging, IAM and regions	CEILING
5	Workloads use federation or managed identity. No long-lived keys	WORKLOAD
6	Joiner, mover and leaver automated. Reviews risk-based with verified revoke	GOVERN

Use Attach each unresolved item to a ticket and an owner. Track closure as a KPI.

Glossary: identity and authentication

Term	Meaning
AAL	Authenticator Assurance Level (NIST SP 800-63B), levels 1 to 3
AiTM	Adversary-in-the-middle proxy that steals session tokens after MFA
CAE	Continuous Access Evaluation, near-real-time revocation of tokens
FIDO2 / WebAuthn	Public-key authentication standards that resist phishing
IdP	Identity provider that authenticates users and issues tokens
OIDC	OpenID Connect, an identity layer on top of OAuth 2.0
PKCE	Proof Key for Code Exchange, protects the authorization code flow
SSO	Single sign-on, one authentication reused across applications

Glossary: authorization, privilege and governance

Term	Meaning
ABAC / RBAC / ReBAC	Attribute, role and relationship based access control
BOLA / BFLA	Broken object-level and function-level authorization
CIEM	Cloud Infrastructure Entitlement Management
IGA	Identity Governance and Administration
JIT / ZSP	Just-in-time access and zero standing privilege
PAM / PAW	Privileged access management and privileged access workstation
PDP / PEP	Policy decision point and policy enforcement point
SoD	Separation of duties, preventing toxic combinations of access

STANDARDS

Standards and specifications to read next

Area	Documents
OAuth and OIDC	RFC 6749, 7636 (PKCE), 8693 (token exchange), 9068, 9449 (DPoP), 9700 (security BCP), OpenID Connect Core
Tokens and provisioning	RFC 7519 (JWT), 8725 (JWT BCP), 7643 and 7644 (SCIM), 8417 (SET), OpenID Shared Signals
Authentication	NIST SP 800-63-4, W3C WebAuthn, FIDO Alliance specifications
Access control	NIST SP 800-162 (ABAC), NIST RBAC model, Google Zanzibar paper, Saltzer and Schroeder (1975)
Zero Trust and controls	NIST SP 800-207, NIST SP 800-53 AC and IA families, CIS Controls v8, OWASP ASVS
Threats	MITRE ATT&CK (Credential Access, Privilege Escalation), OWASP API Security Top 10

Tip Read the security best current practice documents first. They capture the mistakes that shipped and were later fixed.

CLOSING

Start with the identities that can change everything.

Inventory them, remove standing access, protect them with phishing-resistant MFA, log them, and then work outward. Least privilege is a habit that compounds.